Big Data and Hadoop

Complete University Course

Theory, architecture, configuration, practical work, and exam preparation

Abdesslam Rouaijel

How to use this course

This course is designed for an engineering student preparing for an examination. It follows the concepts and terminology in the two practical assignments, distinguishes their Hadoop versions and deployment scopes, and labels additional material as complementary theory. Practical commands are explanatory examples; they are not executed by this document.

Important TP1 / TP2 distinction

TP1's manual installation uses Hadoop 3.3.6 and includes standalone and pseudo-distributed examples with HDFS, YARN and MapReduce. Its optional BDE2020 Docker image is tagged Hadoop 3.1.3/Java 8. TP2 uses Apache Hadoop 3.5.0 with Java 17 in one Ubuntu VM and deploys an HDFS cluster only: one NameNode and two DataNodes initially, with an optional third DataNode. TP2 does not deploy YARN, MapReduce, SecondaryNameNode, HA or Erasure Coding.

Contents

How to use this course i

Architecture diagrams vii

Course scope and practical-work map 1
- 0.1 Coverage matrix . . . 1
- 0.2 How TP1 and TP2 fit together . . . 2
- 0.3 Study priorities . . . 3

1 Introduction to Big Data 4
- 1.1 Data and Big Data . . . 4
- 1.2 The 5 Vs . . . 4
- 1.3 Data organization . . . 5
- 1.4 Traditional processing versus distributed Big Data . . . 5
- 1.5 Distributed computing and parallelism . . . 6
- 1.6 Reliability concepts . . . 6
- 1.7 Batch and stream processing . . . 6
- 1.8 Databases versus distributed storage . . . 7
- 1.9 Applications of Big Data . . . 7

2 Introduction to Hadoop 8
- 2.1 What Hadoop is . . . 8
- 2.2 Why Hadoop was developed . . . 8
- 2.3 Hadoop versus relational databases . . . 9
- 2.4 Hadoop versus Spark . . . 9

2.5 Ecosystem overview . . . 9
2.6 Use-case fit and limits . . . 10

3 HDFS Architecture and Functionality 11
3.1 Purpose and design . . . 11
3.2 Components and metadata . . . 11
3.3 Blocks, block size, replication . . . 12
3.4 Rack awareness and placement . . . 12
3.5 Read operation . . . 12
3.6 Write operation . . . 13
3.7 Failures, fault tolerance and limits . . . 13
3.8 NameNode format: what it does and does not do . . . 13

4 YARN Architecture 14
4.1 YARN components . . . 14

5 MapReduce 15
5.1 MapReduce model . . . 15

6 Hadoop Installation and Configuration 16
6.1 Prerequisites and manual TP1 installation . . . 16
6.2 Starting, stopping, and checking TP1 services . . . 16
6.3 Java and environment . . . 16
6.4 SSH and daemons . . . 17
6.5 Directory concepts . . . 17
6.6 Five important files . . . 17
6.7 core-site.xml . . . 18
6.8 hdfs-site.xml . . . 18
6.9 mapred-site.xml and yarn-site.xml . . . 19
6.10 hadoop-env.sh . . . 20
6.11 TP1/TP2 values side by side . . . 20
6.12 Validate effective configuration, not just file text . . . 20

7 Hadoop Deployment Modes 22
7.1 Three deployment modes . . . 22

8 Hadoop HDFS Commands 23
8.1 Version, startup and process commands . . . 23
8.2 HDFS filesystem shell: hdfs dfs . . . 23
8.3 Administration and diagnostic commands . . . 24
8.4 Linux, Docker and Hadoop command families . . . 25
8.5 TP1 quick validation . . . 25
8.6 Additional requested filesystem commands . . . 25

9 Docker and Hadoop Deployment 27
9.1 Docker concepts . . . 27
9.2 Four layers in TP2 . . . 27
9.3 Compose, networking, ports and names . . . 28
9.4 Bind mounts and persistence . . . 28
9.5 Permissions . . . 29
9.6 TP1 optional Docker approach: BDE2020 image . . . 29

10 Advanced TP2 Configuration **30**
10.1 TP2 operation-by-operation logic . . . 30
10.2 dfs.replication versus hdfs dfs -setrep . . . 31
10.3 Diagnose a wrong NameNode address . . . 32
10.4 Compose structure: compact TP2-aligned example . . . 32
10.5 Representative TP2 command sequence . . . 33

11 Big Data Applications **35**
11.1 What Hadoop does not do by itself . . . 36

12 Complementary Theory **37**
12.1 High Availability, checkpointing and Erasure Coding . . . 37
12.2 HDFS versus a traditional filesystem . . . 37
12.3 Hadoop versus Hive . . . 37
12.4 Important conceptual distinctions . . . 37
12.5 Complementary theory: databases, CAP, partitioning, and ecosystem . . . 38
12.5.1 SQL and NoSQL; relational systems and distributed storage . . . 38
12.5.2 Database versus data warehouse . . . 38
12.5.3 CAP theorem . . . 38
12.5.4 Partitioning versus replication . . . 39
12.5.5 Hadoop ecosystem roles . . . 39
12.5.6 Batch versus real-time processing . . . 39

13 Essential Definitions **40**

14 Comparison Questions and Answers **42**

15 Theoretical Examination Questions **44**
15.1 Fundamentals and Hadoop . . . 44
15.2 HDFS . . . 44
15.3 YARN and MapReduce . . . 45
15.4 Modes and configuration . . . 46
15.5 Docker and TP2 . . . 46
15.6 Operations and applications . . . 47

16 Practical Exercises with Corrections **48**
16.1 Exercise 1 — Count blocks . . . 48
16.2 Exercise 2 — Count replicas . . . 48
16.3 Exercise 3 — Interpret TP2 properties . . . 48
16.4 Exercise 4 — Find wrong TP2 endpoint . . . 48
16.5 Exercise 5 — Distinguish host and container paths . . . 48
16.6 Exercise 6 — Diagnose a duplicate property . . . 49
16.7 Exercise 7 — Understand -setrep . . . 49
16.8 Exercise 8 — YARN versus HDFS . . . 49
16.9 Exercise 9 — Fix malformed XML . . . 49
16.10 Exercise 10 — Read Compose port mapping . . . 49
16.11 Exercise 11 — DataNode container running but report shows 0 live . . . 49
16.12 Exercise 12 — Third DataNode is absent . . . 50
16.13 Exercise 13 — Why does -setrep -w 3 wait? . . . 50
16.14 Exercise 14 — Persistence after compose down . . . 50
16.15 Exercise 15 — Spot a dangerous command . . . 50

17 Troubleshooting Guide **51**

18 Multiple-Choice Questions **53**

19 Final Revision Sheets and Mock Examination **57**
19.1 One-page Hadoop architecture summary . . . 57
19.2 Configuration files cheat sheet . . . 57
19.3 Essential Hadoop commands . . . 58
19.4 Essential Docker commands for TP2 . . . 58
19.5 Ports to memorize . . . 59
19.6 HDFS block and replication calculations . . . 59
19.7 Thirty highest-value examination facts . . . 59
19.8 Common exam traps and misconceptions . . . 60
19.9 Final mock examination (100 points) . . . 61
19.9.1 Part A — Short answers (20 points; 4 points each) . . . 61
19.9.2 Part B — Calculations and interpretation (20 points) . . . 62
19.9.3 Part C — Practical diagnosis (30 points) . . . 62
19.9.4 Part D — Conceptual essay (30 points) . . . 62
19.10 Mock examination correction and marking guidance . . . 62
One-page Big Data summary . . . 64
One-page Hadoop architecture summary . . . 65
One-page HDFS summary . . . 66
One-page Hadoop configuration summary . . . 67
One-page Docker and TP2 summary . . . 68
One-page HDFS and Docker commands summary . . . 69

20 Focused Two-Day Revision Strategy **70**
20.1 Day 1 — Understand the system . . . 70
20.2 Day 2 — Apply and retain . . . 70

21 References **71**
21.1 Assignment sources . . . 71
21.2 Additional official references . . . 71

List of Figures

1 Hadoop separates distributed storage (HDFS), resource management (YARN), and batch computation (MapReduce). . . . vii
2 HDFS clients obtain metadata from the NameNode and transfer file bytes directly with DataNodes. . . . viii
3 For 20 MiB with 8 MiB blocks and replication two: logical blocks are 8, 8, and 4 MiB; six block copies are stored. . . . ix
4 YARN allocates resources through the ResourceManager; an ApplicationMaster coordinates one application across NodeManagers. . . . x

5 WordCount transforms input records to key-value pairs, groups values during shuffle/sort, and sums them in reducers. . . . xi
6 Standalone runs locally; pseudo-distributed mode runs separate daemons on one host; fully distributed mode spreads services across hosts. . . . xii
7 Docker services share a host/VM kernel; volumes or bind mounts persist data outside disposable container layers. . . . xiii
8 TP2 has one Ubuntu VM hosting a private Docker bridge: NameNode RPC is internal on 8020, while the NameNode UI is published on 9870. . . . xiv

List of Tables

1 Coverage matrix reference table . . . 1
2 How TP1 and TP2 fit together reference table . . . 3

1.1 The 5 Vs reference table . . . 4
1.2 Data organization reference table . . . 5
1.3 Traditional processing versus distributed Big Data reference table . . . 5
1.4 Batch and stream processing reference table . . . 6

2.1 What Hadoop is reference table . . . 8
2.2 Hadoop versus relational databases reference table . . . 9
2.3 Ecosystem overview reference table . . . 9

3.1 Components and metadata reference table . . . 11

4.1 YARN components reference table . . . 14

6.1 Five important files reference table . . . 17
6.2 hdfs-site.xml reference table . . . 19
6.3 TP1/TP2 values side by side reference table . . . 20

7.1 Three deployment modes reference table . . . 22

8.1 Version, startup and process commands reference table . . . 23
8.2 HDFS filesystem shell: hdfs dfs reference table . . . 24
8.3 Administration and diagnostic commands reference table . . . 24
8.4 Linux, Docker and Hadoop command families reference table . . . 25
8.5 Additional requested filesystem commands reference table . . . 25

9.1 Docker concepts reference table . . . 27
9.2 Compose, networking, ports and names reference table . . . 28

10.1 TP2 operation-by-operation logic reference table . . . 30
10.2 dfs.replication versus hdfs dfs -setrep reference table . . . 31

11.1 Course reference table . . . 35

12.1 Important conceptual distinctions reference table 38
12.2 Hadoop ecosystem roles reference table 39

17.1 Course reference table 51

19.1 Configuration files cheat sheet reference table 57
19.2 Essential Hadoop commands reference table 58
19.3 Essential Docker commands for TP2 reference table 58
19.4 Ports to memorize reference table 59

Architecture diagrams

Figures 1–8 form a visual atlas for the course. Refer to them alongside the matching chapters: Hadoop architecture (Figure 1), HDFS metadata/data flow (Figure 2), block replication (Figure 3), YARN (Figure 4), MapReduce (Figure 5), deployment modes (Figure 6), generic Docker cluster (Figure 7), and the TP2 network (Figure 8).

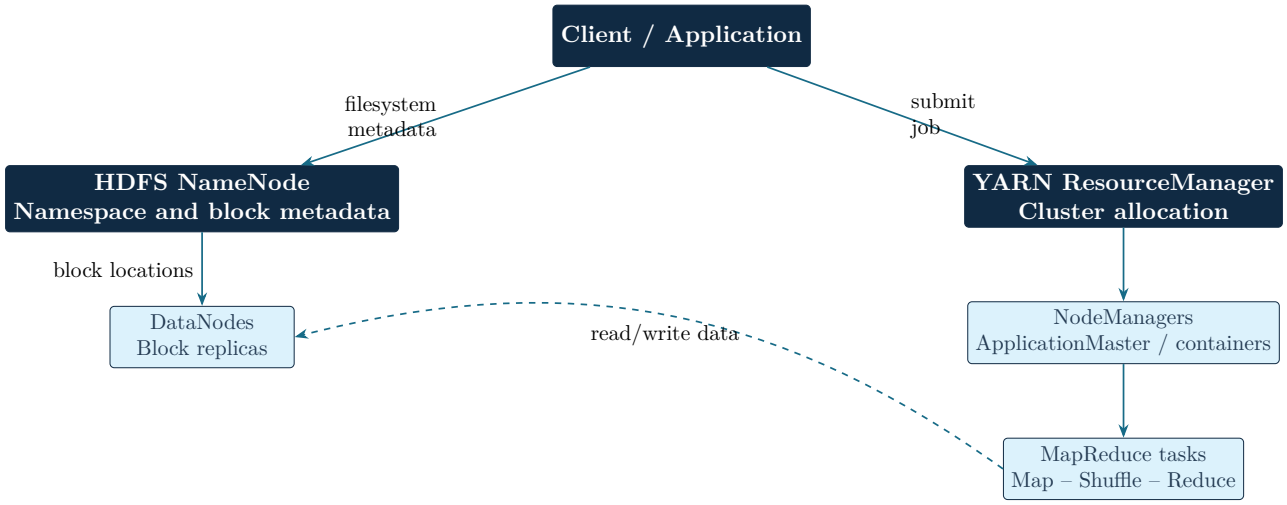


Figure 1: Hadoop separates distributed storage (HDFS), resource management (YARN), and batch computation (MapReduce).

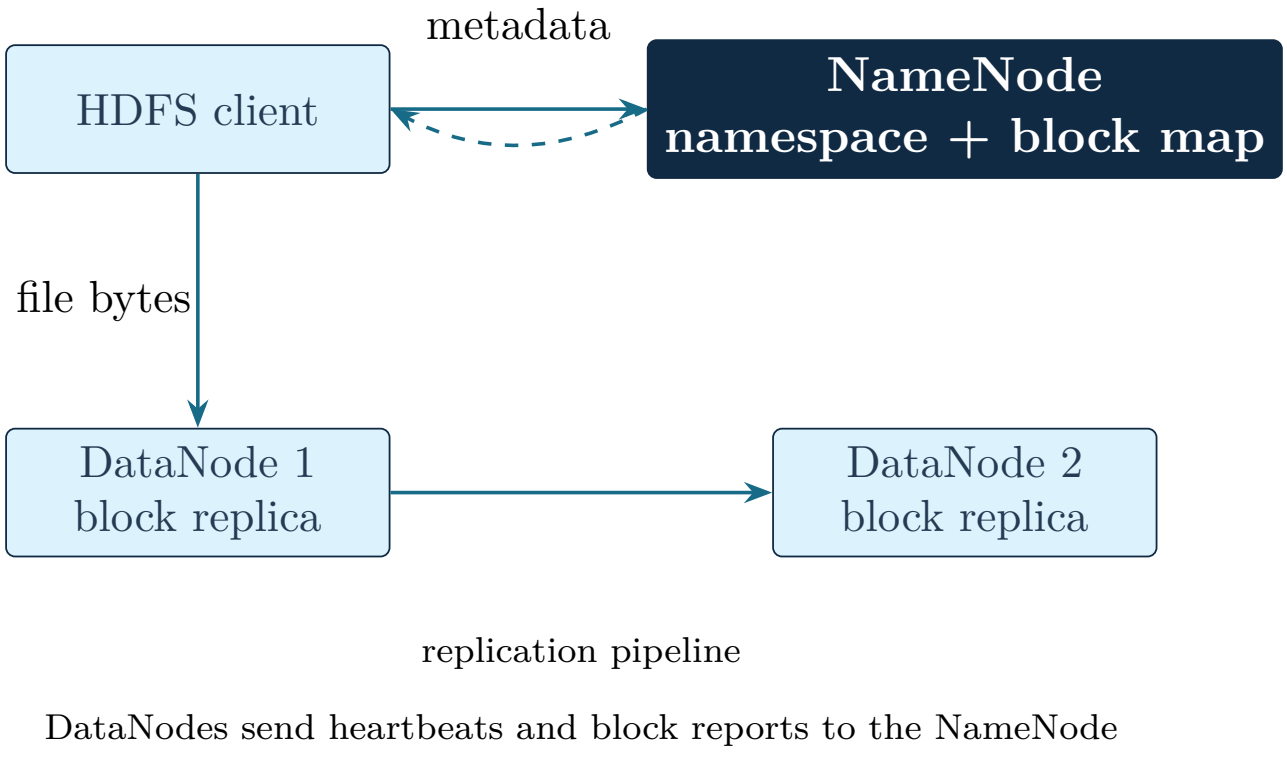


Figure 2: HDFS clients obtain metadata from the NameNode and transfer file bytes directly with DataNodes.

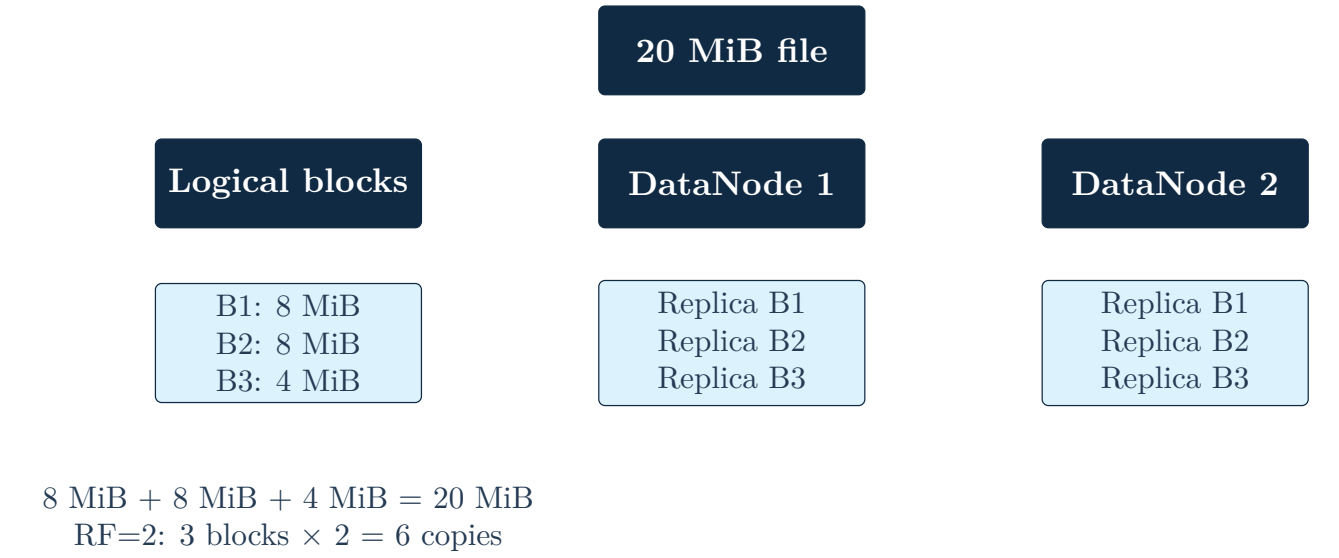
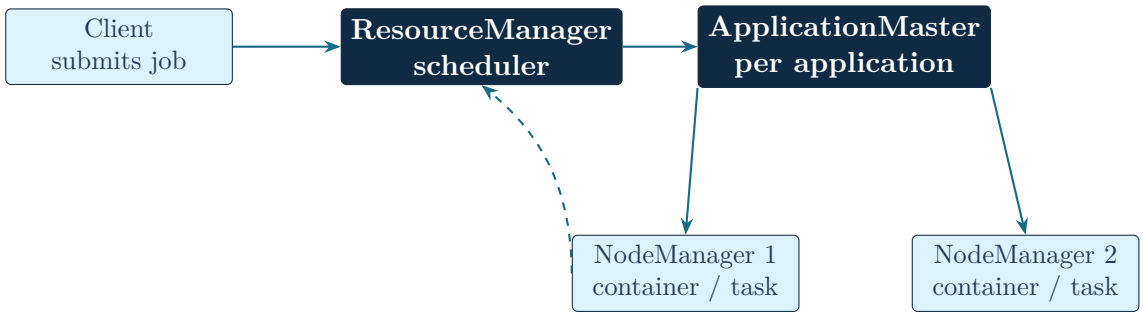


Figure 3: For 20 MiB with 8 MiB blocks and replication two: logical blocks are 8, 8, and 4 MiB; six block copies are stored.



Client submits; ResourceManager launches AM; AM requests and tracks task containers

Figure 4: YARN allocates resources through the ResourceManager; an ApplicationMaster coordinates one application across NodeManagers.



Figure 5: WordCount transforms input records to key-value pairs, groups values during shuffle/sort, and sums them in reducers.

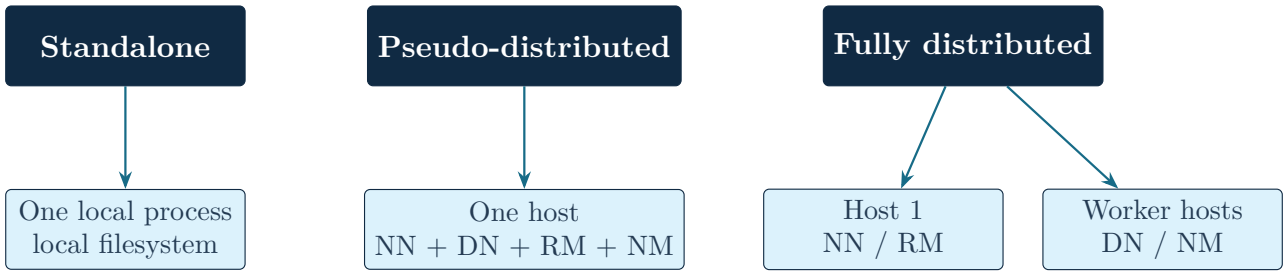


Figure 6: Standalone runs locally; pseudo-distributed mode runs separate daemons on one host; fully distributed mode spreads services across hosts.

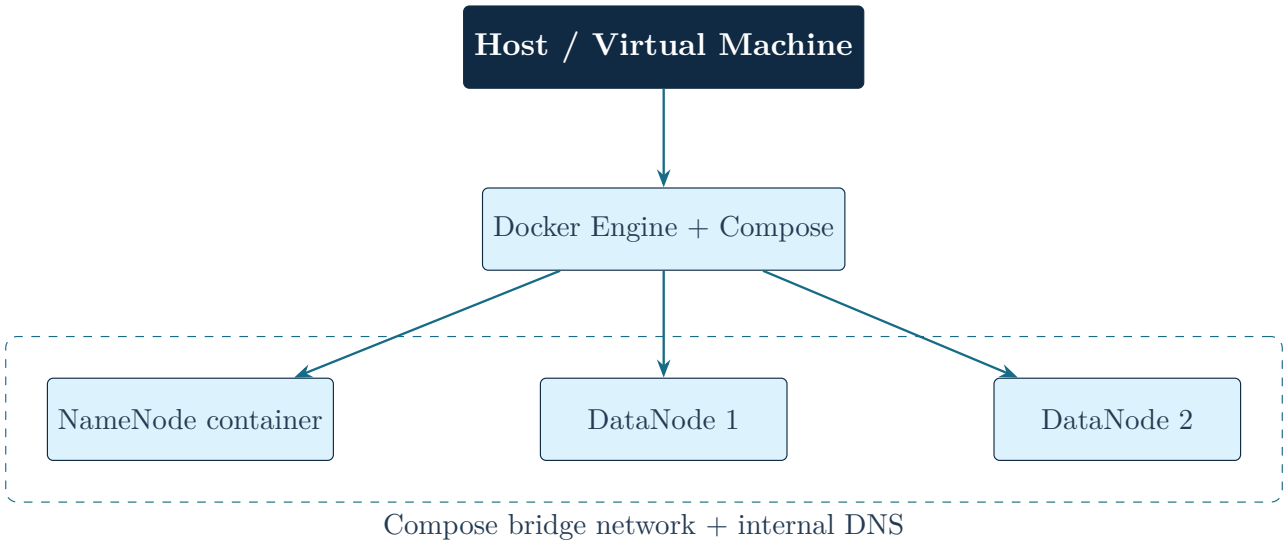


Figure 7: Docker services share a host/VM kernel; volumes or bind mounts persist data outside disposable container layers.

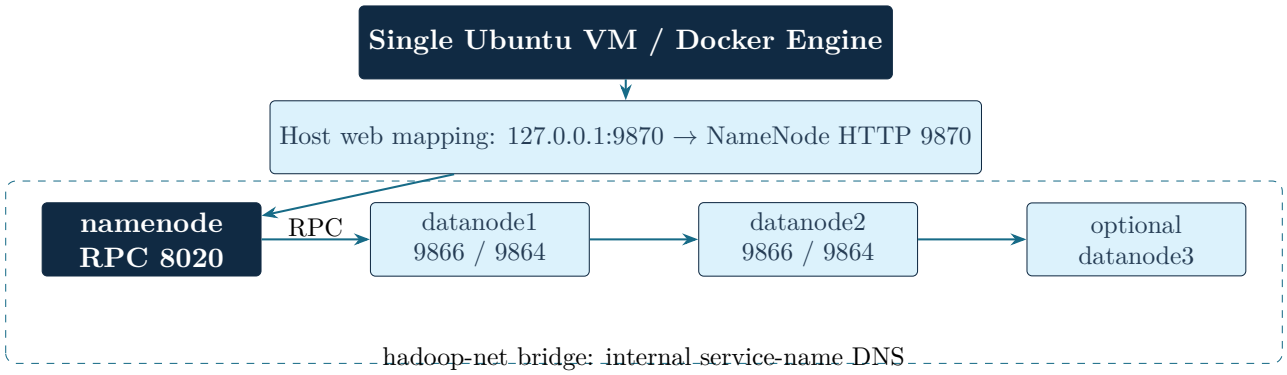


Figure 8: TP2 has one Ubuntu VM hosting a private Docker bridge: NameNode RPC is internal on 8020, while the NameNode UI is published on 9870.

Course scope and practical-work map

0.1 Coverage matrix

Importance estimates are study priorities inferred from the amount of attention and explicit assessment evidence in the assignments, **not predictions of the professor's exam**.

Table 1: Coverage matrix reference table

Coverage matrix — reference table

Topic	TP1 / TP2 evidence	Study focus
Hadoop purpose, modules, HDFS/YARN/MapReduce	TP1: Introduced; TP2: HDFS-focused architecture	Theory: Yes: full component relationships; priority: High
Standalone/local mode	TP1: Hadoop 3.3.6 example and local MapReduce; TP2: Not the deployment mode	Theory: Yes: contrast with cluster modes; priority: High
Pseudo-distributed mode	TP1: Detailed single-host daemon setup; TP2: Not installed	Theory: Yes: process and data-flow detail; priority: High
Fully distributed mode	TP1: Asked as a comparison; TP2: Not deployed	Theory: Yes: multi-host design; priority: High
Java, JAVA_HOME, Hadoop environment	TP1: JDK 8+ prerequisite; HADOOP_HOME, PATH, HADOOP_CONF_DIR; TP2: Java 17 already in image; hadoop-env inspected	Theory: Yes: compatibility and environment scope; priority: High
HDFS NameNode/DataNode and blocks	TP1: Configured and exercised; TP2: Main subject	Theory: Yes: metadata, reads/writes, failure recovery; priority: High
Replication	TP1: TP1 sets 1 in pseudo-distributed example; TP2: Default 2, then explicit 3 replicas	Theory: Yes: placement, under/over-replication; priority: High
Block size	TP1: Not emphasized; TP2: 8 MiB and 20 MiB file calculation	Theory: Yes: production-sized blocks and tradeoffs; priority: High
NameNode format and metadata	TP1: Format command; TP2: One-time format and persistent metadata	Theory: Yes: namespace and edit log; priority: High
Hadoop XML files	TP1: core-site, hdfs-site, mapred-site, yarn-site; TP2: core-site, hdfs-site; hadoop-env inspected	Theory: Yes: properties, precedence, restart behavior; priority: High
HDFS commands and administration	TP1: Basic put/ls/cat; dfsadmin implicitly; TP2: getconf, dfsadmin report, fsck, setrep	Theory: Yes: command families and safe operations; priority: High

Coverage matrix — continued		
MapReduce / YARN	TP1: Modules, config, YARN startup and sample job; TP2: Explicitly not installed in TP2	Theory: Yes: full execution flow; priority: High
Secondary NameNode	TP1: Expected in TP1 jps list; TP2: Explicitly out of TP2 scope	Theory: Yes: checkpoint, not a standby NameNode; priority: Medium
Docker Compose and images	TP1: BDE2020 compose example; TP2: Apache image, services, profiles	Theory: Yes: YAML, networking, lifecycle; priority: High
VM, containers, physical hosts	TP1: Docker Desktop / optional sandbox; TP2: One Ubuntu VM hosting all containers	Theory: Yes: failure domains; priority: High
Docker networking and ports	TP1: NameNode and DataNode ports in sample; TP2: bridge DNS; 8020, 9870, 9866, 9864	Theory: Yes: host/container/network distinction; priority: High
Persistence, mounts, ownership	TP1: Named volumes in example; TP2: Ubuntu bind mounts; UID:GID 1001:100	Theory: Yes: container lifecycle and ownership; priority: High
DataNode extension	TP1: Multiple DataNodes goal; TP2: Compose extension profile activates third	Theory: Yes: adding capacity and re-replication; priority: High
High Availability, Erasure Coding	TP1: Not deployed; TP2: Explicitly out of scope	Theory: Yes, conceptual only; priority: Low/Medium
Big Data Vs, data types, scaling, batch/stream	TP1: Not practically examined; TP2: Not practically examined	Theory: Yes; priority: High
Ecosystem (Hive, HBase, Spark, etc.)	TP1: Names only in Sandbox description; TP2: Not installed	Theory: Yes, roles and boundaries; priority: Medium
Real-world applications	TP1: Not developed; TP2: Not developed	Theory: Yes; priority: Medium

0.2 How TP1 and TP2 fit together

TP1 is a progression of **ways to run Hadoop**:

1. Local/standalone: a Java process uses the local filesystem; useful for code experiments.
2. Pseudo-distributed: several Hadoop daemons run as separate processes on one computer.
3. Docker option: a preconfigured BDE2020 stack reduces manual setup.
4. Example HDFS operations and a MapReduce sample.

TP2 is a deeper **HDFS deployment and diagnostic exercise**. It uses Ubuntu inside one VirtualBox VM, Docker Engine, and Apache's `apache/hadoop:3.5.0` image. Two DataNodes start initially; a Compose profile adds the third. It examines effective configuration, internal DNS, port publishing, storage mounts and permissions, block placement, replication changes, and persistence.

Keep the versions distinct:

Table 2: How TP1 and TP2 fit together reference table

How TP1 and TP2 fit together — reference table		
Context	Hadoop / Java	What the assignment uses
TP1 manual installation	Hadoop 3.3.6; JDK 8 or later stated	Standalone and pseudo-distributed examples
TP1 Docker sample	BDE2020 image `2.0.0-hadoop3.1.3-java8`	Optional ready-made cluster; it is not Hadoop 3.3.6
TP2	Apache Hadoop 3.5.0; Java 17 in image	HDFS-only cluster inside Ubuntu VM

TP2 does **not** deploy YARN, MapReduce, SecondaryNameNode, HA, Hive, or Spark. Explain these as theory; do not describe them as running services in TP2.

0.3 Study priorities

First master HDFS roles, block arithmetic, replication and the relationship between XML and effective configuration. Next master TP2's Docker/VM topology, ports, bind mounts, ownership, NameNode formatting, and the difference between default replication and `-setrep`. Then review Hadoop operating modes, TP1 configuration and YARN/MapReduce theory. Finally, learn the wider ecosystem and advanced concepts.

Introduction to Big Data

Figure 1 previews distributed storage and computation.

1.1 Data and Big Data

Data are recorded representations of observations or events: numbers, text, images, sensor readings, transactions, audio, or logs. Data become useful when processed and interpreted in a context.

Big Data describes data and workloads whose scale, arrival rate, diversity, or quality challenges the capabilities of traditional single-machine tools and processes. It is not defined by one universal number of bytes. A dataset is "big" relative to the storage, compute, latency, and governance requirements of the organization.

- **Simple explanation:** A spreadsheet is sufficient until the data or the speed of analysis exceeds what one machine and its software can handle reliably.
- **Technical view:** Big Data systems partition data and work, coordinate distributed resources, recover from partial failure, and often combine batch and streaming pipelines.
- **Example:** A telecom operator collects millions of call-detail and network events per hour. It may retain raw events in a distributed store, aggregate them by region, and alert on anomalies.
- **Exam-ready:** "Big Data is data whose volume, velocity, variety and other operational properties require scalable, distributed methods for acquisition, storage, processing and analysis."
- **Common confusion:** Big Data is not simply "a very large database," and Hadoop is not the definition of Big Data.

1.2 The 5 Vs

Table 1.1: The 5 Vs reference table

The 5 Vs — reference table			
V	Meaning	Example	Technical challenge
Volume	Amount of data	Years of clickstream events	Distributed storage and parallel computation
Velocity	Rate of generation/ arrival and required response speed	Payment events arriving continuously	Ingestion, queues, low-latency processing
Variety	Different formats and schemas	JSON logs, relational rows, images	Parsing, schema evolution, integration
Veracity	Reliability, uncertainty, completeness and bias	Duplicate or missing sensor readings	Validation, cleaning, provenance

The 5 Vs — continued			
Value	Useful benefit derived from data	Detecting equipment faults early	Ask a decision-relevant question; control cost

The 5 Vs are a useful framework, not a formal requirement that every dataset must have all five properties to the same degree. Some courses add further Vs such as variability, but the five above are the requested core set.

1.3 Data organization

Table 1.2: Data organization reference table

Data organization — reference table			
Type	Description	Example	Typical handling
Structured	Fixed fields and a defined schema	SQL table of orders	Relational queries; export to files if needed
Semi-structured	Self-describing tags/keys, flexible schema	JSON event, XML document	Parse fields; schema can vary between records
Unstructured	No simple tabular schema	Image, audio, free-form text	Store as objects/files; extract features or metadata

"Unstructured" does not mean "no structure at all": images have pixels and audio has samples, but their meaning is not naturally a set of fixed relational columns.

1.4 Traditional processing versus distributed Big Data

Table 1.3: Traditional processing versus distributed Big Data reference table

Traditional processing versus distributed Big Data — reference table		
Dimension	Traditional/single-node approach	Distributed approach
Storage	One server or scale-up array	Data split across machines
Compute	One machine or tightly coupled server	Tasks run in parallel on a cluster
Growth	Often add CPU/RAM to one system	Add nodes (horizontal scaling)
Failure	Whole-machine failure may stop service	Replicas/retries can tolerate some node failures
Design cost	Simpler for modest data	Network, coordination, consistency and operations add complexity
Best fit	Transactional, interactive, moderate workloads	Large scans, batch analytics, high-volume ingest

Distributed systems do not automatically make every workload faster. Network transfer, serialization, startup and coordination have costs; small jobs may be faster on one computer.

1.5 Distributed computing and parallelism

A distributed system uses multiple networked computers that coordinate to provide a service. Parallel processing divides a computation into pieces that can run at the same time.

Example: For a word count over 1,000 files, independent file splits can be mapped on different workers. Partial counts for a word are then combined. The final result still requires coordination and data transfer (the shuffle).

- **Data parallelism:** same operation on different partitions.
- **Task parallelism:** different operations execute concurrently.
- **Scale-out / horizontal scaling:** add machines or workers.
- **Scale-up / vertical scaling:** give one machine more CPU, memory, or storage.

1.6 Reliability concepts

- **Fault tolerance:** the service can continue or recover correctly despite specified component failures.
- **Availability:** the proportion of time a service is usable.
- **Durability:** probability that stored data remain intact over time.
- **Replication:** keep multiple copies on separate storage nodes; this can preserve availability after some failures.
- **Backup:** create a separately managed recovery copy, usually with retention and isolation from the live system.

Replication is not a backup: accidental deletion or corruption may be propagated to all live replicas. HDFS replication protects against certain disk/node failures, not every operational or site-wide disaster.

1.7 Batch and stream processing

Table 1.4: Batch and stream processing reference table

Batch and stream processing — reference table	
Batch	Stream
Process a bounded collection, often on a schedule	Process unbounded events as they arrive
High throughput; results may be delayed	Low-latency results; state and ordering matter
Example: nightly report over yesterday's logs	Example: alert on a suspicious payment in seconds
Hadoop MapReduce is a classic batch model	Kafka Streams, Flink, or Spark Structured Streaming are common stream technologies

A system may use both: a streaming path for immediate alerts and a batch path to recompute accurate historical reports.

1.8 Databases versus distributed storage

A relational database offers schemas, indexes, transactions and low-latency record operations. HDFS is a distributed file system optimized for high-throughput access to large files and batch scans; it is not a general replacement for an OLTP database. A data lake may store files in HDFS or object storage, while databases and query engines provide other access patterns.

1.9 Applications of Big Data

- **Industry/IoT:** sensor histories support predictive maintenance.
- **Healthcare:** de-identified records and imaging metadata support population studies; privacy and governance are essential.
- **Finance:** event histories support fraud analysis and risk reporting.
- **Transport:** GPS and traffic observations support route and demand analysis.
- **Telecommunications:** call-detail and network telemetry support capacity planning.
- **AI:** large curated datasets can train models; a storage/processing platform prepares data but is not itself the learning algorithm.

Introduction to Hadoop

Refer to Figure 1 for the platform view.

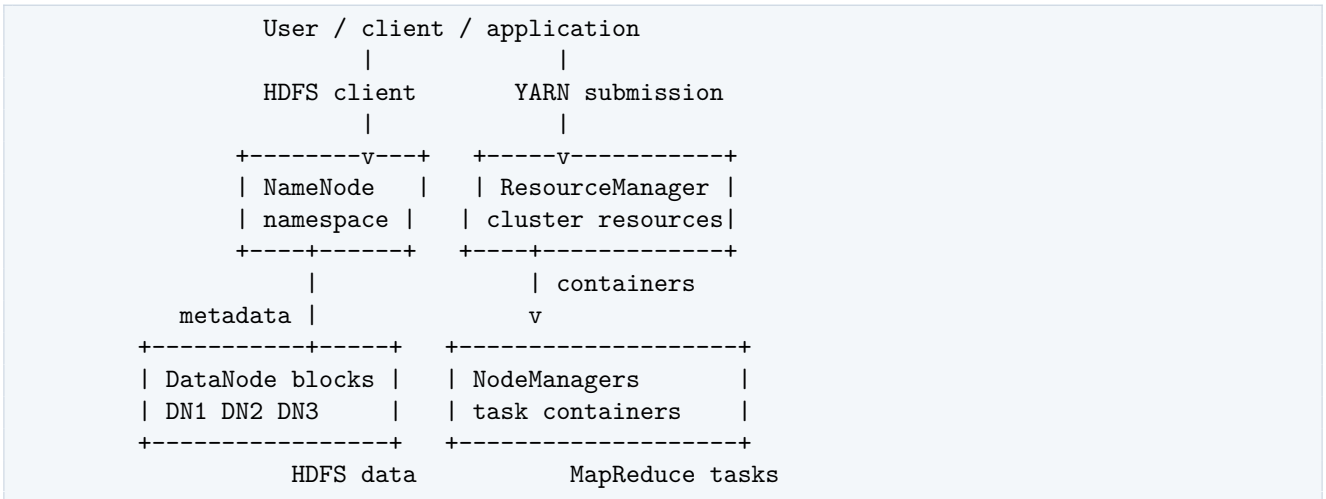
2.1 What Hadoop is

Apache Hadoop is an open-source framework and set of modules for distributed storage and processing of large datasets on clusters. The four core modules are:

Table 2.1: What Hadoop is reference table

What Hadoop is — reference table		
Module	Main responsibility	Key distinction
Hadoop Common	Shared libraries and utilities	Foundation used by other modules
HDFS	Distributed file storage	Stores namespace metadata and file blocks
YARN	Cluster resource management and scheduling	Allocates resources to applications
MapReduce	Batch computation model and execution framework	Runs map/shuffle/reduce tasks, often on YARN

Architecture sketch



In practice, the NameNode manages filesystem metadata while clients transfer file bytes directly to/from DataNodes. YARN allocates containers; an ApplicationMaster coordinates one submitted application.

2.2 Why Hadoop was developed

Large web-scale workloads needed a way to store and scan very large datasets across many commodity machines, while expecting individual machines to fail. Hadoop brought distributed storage (HDFS) and

distributed batch processing (MapReduce) into an open ecosystem, with computation scheduled close to data when possible.

2.3 Hadoop versus relational databases

Table 2.2: Hadoop versus relational databases reference table

Hadoop versus relational databases — reference table	
Hadoop/HDFS + processing engine	Relational database
Distributed files and large sequential scans	Tables, indexes and record-oriented access
Often schema-on-read for data lake files	Schema commonly enforced on write
High throughput, usually higher query latency	Interactive transactions and indexed lookups
HDFS replication and distributed task execution	Database-specific availability, transaction and replication mechanisms
Not inherently an OLTP database	Designed for structured queries and transactions

These are broad tendencies, not absolute limits; modern database and lakehouse systems overlap.

2.4 Hadoop versus Spark

Hadoop is a project/ecosystem, not synonymous with MapReduce. Spark is a separate distributed compute engine that can run on YARN and read HDFS. Classic MapReduce materializes intermediate results between stages; Spark can cache data in memory and offers APIs for batch, SQL, streaming and ML workloads. Spark still needs storage and cluster resources and does not replace HDFS merely by being a compute engine.

2.5 Ecosystem overview

Table 2.3: Ecosystem overview reference table

Ecosystem overview — reference table	
Project	Role (conceptual)
Hive	SQL-like data warehouse/query layer over distributed data
Pig	Data-flow scripting, historically used for ETL
HBase	Distributed column-family NoSQL database, often on HDFS
Sqoop	Historical bulk transfer between relational databases and Hadoop
Flume	Historical/legacy log and event ingestion system
Oozie	Historical workflow scheduler for Hadoop jobs
ZooKeeper	Coordination service used by distributed systems
Spark	General distributed compute engine; can use HDFS/YARN

These ecosystem tools are not all part of Hadoop's four core modules, and several have evolved in adoption or maintenance. TP1 mentions ecosystem tools in the context of preconfigured sandbox distributions; TP2 installs none of them.

2.6 Use-case fit and limits

Good fit: large datasets, high-throughput scans, ETL, log processing and batch aggregation. Less suitable as the only solution for tiny datasets, millisecond interactive lookups, strongly transactional workloads, or workloads whose coordination cost dominates. Hadoop clusters also require administration, security, monitoring, data governance and capacity planning.

Course note

Exam-ready answer: Hadoop is an open-source framework for distributed storage and processing. HDFS stores files as replicated blocks, YARN manages cluster resources, MapReduce provides a batch processing model, and Hadoop Common supplies shared utilities. HDFS, YARN and MapReduce solve different but complementary problems.

HDFS Architecture and Functionality

Figures 2 and 3 illustrate data flow and block placement.

3.1 Purpose and design

HDFS is a distributed filesystem designed for large files, high-throughput streaming access, and recovery from common node/storage failures. It favors throughput over low-latency random updates. Its common model is write-once/read-many, with append support; it is not a POSIX local disk and not intended for many tiny files or frequent in-place modifications.

- **Simple explanation:** HDFS divides a large file into blocks and stores copies of those blocks on different worker machines.
- **Technical explanation:** The NameNode manages the namespace and block-to-DataNode mapping. DataNodes store block files, report heartbeats and block reports, and serve client data. The client asks the NameNode for metadata, then exchanges bytes with DataNodes.
- **Example:** TP2 stores a 20 MiB test file in HDFS using 8 MiB blocks and replication factor 2.
- **Exam-ready:** "HDFS is Hadoop's distributed filesystem. A NameNode manages namespace metadata and DataNodes store and serve replicated file blocks."
- **Common confusion:** The NameNode does not normally relay the file's contents; it provides metadata and coordination.

3.2 Components and metadata

Table 3.1: Components and metadata reference table

Components and metadata — reference table		
Component	Responsibility	TP context
NameNode	Namespace, permissions, file-to-block mapping, replica locations and coordination	One container in TP2
DataNode	Stores block replicas; serves reads/writes; reports status	Two initially, third in extension
HDFS client	Contacts NameNode for metadata, then DataNodes for bytes	`hdfsdfs...` client in container
SecondaryNameNode	Periodically checkpoints namespace state by merging fsimage and edits	Expected by TP1 `jps` in pseudo-distributed setup; not deployed in TP2

HDFS metadata includes paths, file ownership/permissions, block IDs, replication targets and mappings from blocks to DataNodes. The actual user file bytes are stored in DataNode block files.

SecondaryNameNode warning: Its name is misleading. It is not a real-time backup, hot standby, or automatic replacement for a failed NameNode. HDFS High Availability uses an Active/Standby arrangement and shared edit-log coordination (for example JournalNodes), typically with failover coordination.

3.3 Blocks, block size, replication

An HDFS file is split into logical blocks. All full blocks have the configured block size; the final block is only as large as the remaining data. Each block is stored as one or more replicas according to the file's replication factor. The NameNode tracks metadata, not copies of all data bytes.

TP2 block calculation

Assume the file is exactly **20 MiB**, block size is **8 MiB** (8,388,608 bytes), and replication factor is **2**:

```
20 MiB = 8 MiB + 8 MiB + 4 MiB
logical blocks = ceil(20 / 8) = 3
expected block replicas = 3 blocks x 2 = 6 block copies
```

The last block is 4 MiB; HDFS does not pad it to 8 MiB. With two live DataNodes, each logical block should have one copy on each DataNode when placement constraints permit. After a third DataNode is live and the file is changed to replication 3, each of the three blocks can have three copies, normally distributed across the three DataNodes. Actual placements are decided by HDFS and should be verified with `hdfsfsck-files-blocks-locations`, not guessed.

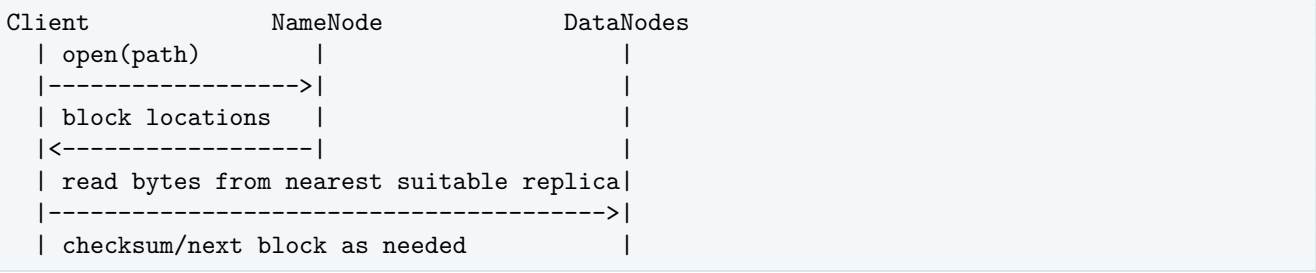
Units: TP2's `ddbs=1Mcount=20` creates 20 MiB in binary units (20 × 1,048,576 bytes). "20 MB" is sometimes used ambiguously for decimal MB; use the actual byte count for exact calculations.

Production trade-off: TP2's 8 MiB is intentionally small for classroom observation. Many HDFS deployments historically use 128 MiB or larger blocks. Larger blocks reduce per-block metadata and improve large sequential scans, but the right value depends on workload and deployment.

3.4 Rack awareness and placement

Rack awareness allows the NameNode to reason about rack/network topology when placing replicas. Distributing copies across racks can reduce the risk of losing every replica in a rack outage and balance network use. It does not mean every pair of copies is necessarily on different physical hosts/racks in every small cluster. TP2 containers share a single VM and are not separate physical failure domains.

3.5 Read operation



The client requests file metadata and ordered block locations from the NameNode. It chooses a suitable replica (often the closest network location), reads directly from a DataNode, validates checksums, and proceeds to the next block. If a replica is unavailable or corrupt, the client can try another known replica.

3.6 Write operation

```
Client -> NameNode: create file; check permissions/existence
NameNode -> Client: target DataNodes for first block
Client -> DN1 -> DN2 [-> DN3]: stream block through replication pipeline
DataNodes -> Client: acknowledgements flow back through pipeline
Client repeats for next block, then closes file
NameNode records completed file/block metadata
```

The NameNode authorizes and allocates block targets; file data is pipelined among DataNodes. A write succeeds only after acknowledgements meet protocol requirements. HDFS enforces one writer per file and generally does not support arbitrary in-place updates.

3.7 Failures, fault tolerance and limits

- **DataNode failure:** missed heartbeats lead the NameNode to mark it unavailable; the NameNode identifies under-replicated blocks and schedules new replicas if enough live nodes and data remain. A client may read another replica.
- **Disk failure:** affected block copies become unavailable/corrupt; checksums help detect corruption, and good replicas can be used to repair.
- **NameNode failure:** without HA, namespace operations are unavailable until recovery/restart from persistent metadata. Data blocks may still physically exist, but clients cannot reliably resolve file paths. A SecondaryNameNode checkpoint alone does not provide automatic failover.
- **HDFS advantages:** scale-out capacity/throughput, large-file support, replication and automatic recovery from some failures.
- **Limitations:** NameNode metadata memory/availability matters; small files create metadata overhead; latency is not optimized for random interactive access; replication consumes storage; HDFS is not a backup or database.

3.8 NameNode format: what it does and does not do

`hdfsnamenode-format` initializes a **new HDFS namespace**, creating metadata such as an FSImage, initial edits and a cluster identity in the configured NameNode directory. It does not format the Ubuntu disk. In TP2 it is run once before first startup, against `data/namenode` mounted at `/data/hdfs/name`.

Never repeat formatting on a cluster containing needed data. Reinitializing NameNode metadata can disconnect existing DataNode blocks from their namespace and make files inaccessible. Restarting or recreating a container with persistent metadata does not require formatting.

YARN Architecture

Figure 4 summarizes YARN execution roles.

4.1 YARN components

YARN separates cluster resource management from individual processing frameworks.

Table 4.1: YARN components reference table

YARN components — reference table	
Component	Role
ResourceManager (RM)	Cluster-wide authority; accepts applications and allocates resources through a scheduler
NodeManager (NM)	Runs on a worker; launches/monitors containers and reports resources/ health
ApplicationMaster (AM)	One per application; negotiates containers and coordinates that application's work
Container	Resource allocation (e.g. memory and vcores) in which an application task/ process runs

```
Client submits application
          |
          v
ResourceManager -- grants first container --> ApplicationMaster
          ^                                        |
          | resource reports/requests             | requests task containers
          +------------ NodeManagers <-------------+
                        run containers/tasks
```

The scheduler allocates resources; it is not itself the application task monitor. The ApplicationMaster handles application-specific coordination. A container is a resource allocation/execution environment, not necessarily a Docker container.

MapReduce

Figure 5 visualizes the WordCount pipeline.

5.1 MapReduce model

MapReduce is a distributed batch computation model:

1. **Input/splitting:** input files are divided into logical input splits. RecordReaders turn bytes into records.
2. **Map:** each mapper transforms records into intermediate key-value pairs.
3. **Combiner (optional):** local aggregation can reduce shuffle traffic; it must be valid for the operation and is not guaranteed to run.
4. **Partition/shuffle/sort:** intermediate keys are partitioned by reducer, transferred across the network, grouped and sorted by key.
5. **Reduce:** each reducer processes one key and its values to produce output pairs.
6. **Output:** results are written to HDFS, commonly one output file per reducer. The output directory must not already exist for the standard job output committer.

Hadoop Installation and Configuration

6.1 Prerequisites and manual TP1 installation

TP1 uses Ubuntu (or Windows through WSL2/Docker Desktop), a JDK, configured `JAVA_HOME`, and network access. The manual installation uses Hadoop **3.3.6**. Its optional BDE2020 Docker image is a separate build tagged Hadoop **3.1.3 / Java 8**.

```
java -version
sudo apt update
sudo apt install openjdk-8-jdk
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
sudo mv hadoop-3.3.6 /usr/local/hadoop
```

Set `JAVA_HOME` in `etc/hadoop/hadoop-env.sh`, then configure the shell:

```
export HADOOP_HOME=/usr/local/hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export HADOOP_CONF_DIR=$HADOOP_HOME/etc/hadoop
source ~/.bashrc
hadoop version
```

A successful `hadoopversion` confirms the client installation, not that HDFS daemons are running. Before pseudo-distributed startup, TP1 configures passwordless SSH to `localhost`, sets the XML properties, and formats the NameNode once for a new namespace. The exact URI and data paths differ from TP2.

6.2 Starting, stopping, and checking TP1 services

For a new empty namespace only, `hdfsnamenode-format` initializes metadata. Then `start-dfs.sh` starts HDFS and `start-yarn.sh` starts YARN; stop with `stop-dfs.sh` and `stop-yarn.sh`. TP1 verifies Java daemons with `jps`, HDFS at `http://localhost:9870`, and the ResourceManager at `http://localhost:8088`. `jps` is only a process check; confirm worker registration with service-level reports. For containerized Hadoop, use Docker Compose and container logs instead of host startup scripts.

6.3 Java and environment

- `JAVA_HOME`: root of the Java installation used by Hadoop scripts/daemons. TP1 requests JDK 8 or later; TP2 uses Java 17 included in the Apache image.
- `HADOOP_HOME`: Hadoop installation root in TP1 (example `/usr/local/hadoop`).
- `HADOOP_CONF_DIR`: directory containing Hadoop configuration; TP1 example `$HADOOP_HOME/etc/hadoop`. TP2 sets `/opt/hadoop/etc/hadoop` in containers.
- `PATH`: enables commands such as `hadoop`, `hdfs`, `start-dfs.sh`, `stop-dfs.sh`.

- Hadoop version and Java compatibility must be checked for the exact release and distribution. Do not infer that every Java version is supported merely because `java-version` runs.

`hadoop-env.sh` is a shell configuration script for Hadoop launchers, environment values and JVM options. It is not an XML file and is distinct from `core-site.xml`.

6.4 SSH and daemons

TP1 uses passwordless SSH to `localhost` so scripts such as `start-dfs.sh` can launch local daemons through SSH. In multi-host deployments, secure remote management and authentication are operational requirements; SSH is not a replacement for Hadoop service security (Kerberos in secured deployments).

`jps` lists Java processes visible to the current user. In TP1's pseudo-distributed example, expected processes include `NameNode`, `DataNode`, `SecondaryNameNode`, `ResourceManager`, `NodeManager`. A process listing is a first check, **not proof** that DataNodes are live or HDFS works. TP2 instead checks `dockercomposeps`, container logs, and `hdfsdfsadmin-report`.

6.5 Directory concepts

The Hadoop install directory contains binaries/configuration. HDFS NameNode and DataNode data directories contain runtime metadata and blocks. Keep these separate. The value in a config file must match the directory mounted into a container (TP2 maps Ubuntu `data/namenode` to `/data/hdfs/name` and each DN folder to its internal data path).

Hadoop configuration uses XML property pairs. A configuration file must be well-formed XML; property names must be spelled correctly; the service/client must read the intended config directory. A syntactically valid but ineffective file can still leave Hadoop using a default or another configuration.

6.6 Five important files

Table 6.1: Five important files reference table

Five important files — reference table

File	Main purpose	Main scope	TP use
`core-site.xml`	Common filesystem/client configuration	Hadoop-wide/common	TP1 `fs.defaultFS=hdfs://localhost:9000`; TP2 `hdfs://namenode:8020`
`hdfs-site.xml`	HDFS storage, replication, block and service properties	HDFS	TP1 replication 1/directories; TP2 replication, block size, paths and ports
`mapred-site.xml`	MapReduce framework and runtime configuration	MapReduce	TP1 `mapreduce.framework.name=yarn`
`yarn-site.xml`	YARN and NodeManager configuration	YARN	TP1 `mapreduce_shuffle` auxiliary service

Five important files — continued		
`hadoop-env.sh`	Environment and Hadoop JVM launch options processes	TP1 Java path/environment; TP2 inspected, not changed

6.7 core-site.xml

`fs.defaultFS` defines the default filesystem URI used when a path has no explicit scheme/authority.

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://namenode:8020</value>
  </property>
</configuration>
```

URI breakdown:

```
hdfs://namenode:8020
^^^^   ^^^^^^^^ ^^^^
scheme hostname port
```

In TP2, Docker's internal DNS resolves `namenode` on the Compose bridge. `8020` is the NameNode RPC endpoint. The RPC port need not be published to Ubuntu for container peers on the same Docker network. TP1 pseudo-distributed example uses `hdfs://localhost:9000`; do not substitute that TP1 value into TP2.

6.8 hdfs-site.xml

TP2 configuration (as supplied in the assignment):

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property><name>dfs.replication</name><value>2</value></property>
  <property><name>dfs.blocksize</name><value>8388608</value></property>
  <property><name>dfs.namenode.name.dir</name><value>file:///data/hdfs/name</value></property
    >
  <property><name>dfs.datanode.data.dir</name><value>file:///data/hdfs/data</value></property
    >
  <property><name>dfs.namenode.rpc-address</name><value>namenode:8020</value></property>
  <property><name>dfs.namenode.http-address</name><value>0.0.0.0:9870</value></property>
  <property><name>dfs.datanode.address</name><value>0.0.0.0:9866</value></property>
  <property><name>dfs.datanode.http.address</name><value>0.0.0.0:9864</value></property>
</configuration>
```

The compact one-line properties above are valid XML; line-breaking each property is often easier to read and debug.

Table 6.2: hdfs-site.xml reference table

hdfs-site.xml — reference table			
Property	Purpose / effect	Component	Restart/recreate?
`dfs.replication`	Default replication for newly created files when not overridden; does not retroactively change existing files	HDFS	Recreate/restart services or client processes that must load changed config; use `-setrep` for existing file
`dfs.blocksize`	Default block size for newly created files; file metadata records the block layout	HDFS	Restart/recreate for new config; existing files retain their layout
`dfs.namenode.name.dir`	Local path(s) where NameNode namespace metadata is persisted	NameNode	Set before initialization; changing after format points at a different/ empty namespace
`dfs.datanode.data.dir`	Local path(s) where DataNode block data are persisted	DataNode	Restart/recreate; path must be writable and correctly mounted
`dfs.namenode.rpc-address`	Host/port used for NameNode RPC (especially relevant to explicitly named/HA configurations)	NameNode/client config	Restart/recreate
`dfs.namenode.http-address`	HTTP bind address/port for NameNode web UI	NameNode	Restart/recreate; Compose host mapping is a separate layer
`dfs.datanode.address`	DataNode data-transfer listener address/port; 9866 in TP2	DataNode	Restart/recreate
`dfs.datanode.http.address`	DataNode HTTP listener address/ port; 9864 in TP2	DataNode	Restart/recreate

TP2 uses `0.0.0.0` inside the container to bind the service on its interfaces; this is not the same as publishing every port to the host. A firewall/port mapping controls host access separately.

6.9 mapred-site.xml and yarn-site.xml

```
<!-- mapred-site.xml -->
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

This selects YARN as the framework for MapReduce jobs. It does not install YARN or start its daemons by itself.

```
<!-- yarn-site.xml -->
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

`mapreduce_shuffle` enables the NodeManager shuffle service required for MapReduce reducers to fetch mapper outputs. Again, the property does not itself start a NodeManager.

6.10 hadoop-env.sh

Example:

```
export JAVA_HOME=/path/to/supported/java
```

This is shell syntax. `JAVA_HOME` tells launch scripts which Java installation to use. JVM options can tune process memory/behavior. Changes generally affect newly started Hadoop processes, so restart/recreate affected daemons. In TP2, Java 17 is included in the image and the assignment explicitly says not to change the working path without reason.

6.11 TP1/TP2 values side by side

Table 6.3: TP1/TP2 values side by side reference table

TP1/TP2 values side by side — reference table		
Property	TP1 pseudo-distributed example	TP2
`fs.defaultFS`	`hdfs://localhost:9000`	`hdfs://namenode:8020`
`dfs.replication`	`1`	Initially `2`, then experiment with existing file `-setrep3`, and config default `3` for new file
`dfs.blocksize`	Not explicitly set	`8388608` bytes (8 MiB)
NameNode/data paths	`/usr/local/hadoop/hdfs/...`	Container paths under `/data/hdfs/...`, bind-mounted to Ubuntu
Execution	HDFS + YARN configuration example	HDFS only

6.12 Validate effective configuration, not just file text

Use the assignment's checks:

```
docker exec tp2-namenode hdfs getconf -confKey fs.defaultFS
docker exec tp2-namenode hdfs getconf -confKey dfs.replication
docker exec tp2-namenode hdfs getconf -confKey dfs.blocksize
```

```
docker exec tp2-namenode hdfs getconf -confKey dfs.namenode.name.dir
```

Expected initial TP2 values: `hdfs://namenode:8020`, `2`, `8388608`, `file:///data/hdfs/name`. If a value is stale, check XML path/mount, `HADOOP_CONF_DIR`, duplicate property entries, then recreate/restart the process that reads it. `getconf` reports the client's loaded configuration; service logs and behavior remain important evidence that the daemon is operating with the intended settings.

Hadoop Deployment Modes

Figure 6 contrasts the three modes.

7.1 Three deployment modes

Table 7.1: Three deployment modes reference table

Three deployment modes — reference table				
Mode	Machines/processes	Configuration	Best for	Limitation
Standalone (local)	One machine; normally one Java process; local filesystem; no HDFS daemons	Minimal/default local mode	Learning, debugging, local MapReduce examples (TP1 Hadoop 3.3.6)	No distributed storage or cluster fault tolerance
Pseudo-distributed	One machine; Hadoop daemons run as separate processes, often via SSH scripts	XML config; HDFS directories; SSH; format before first use	Learn daemon interactions on one computer; TP1's 1 NN/ 1 DN and YARN example	Processes share one host and failure domain
Fully distributed	Multiple networked machines; NN/RM and worker services distributed	Hostnames, network, storage, security, resource and HA design	Real cluster and scale-out	Operational complexity; needs production security/ monitoring

Avoid a common wording trap: "single-node" may refer to a pseudo-distributed cluster on one machine; standalone mode is not the same thing. A local MapReduce job can be run in different configurations; identify both storage mode and execution framework.

Hadoop HDFS Commands

Course note

Commands are examples, not a request to run them. Run them in the appropriate environment and project directory. TP2 warns to preserve `data/` and never reformat an initialized cluster.

8.1 Version, startup and process commands

Table 8.1: Version, startup and process commands reference table

Version, startup and process commands — reference table		
Command	Meaning / example	Expected behavior and common mistake
`hadoopversion`	Print Hadoop distribution/ version, e.g. TP1 expects 3.3.6 for its manual install	Does not prove a cluster is running; on TP1 Docker image it may be 3.1.3, TP2 image 3.5.0
`hdfsversion`	Print HDFS/Hadoop version information	Version check only, not a connectivity test
`hdfsnamenode-format`	Initialize a new NameNode namespace	Use only once on a new cluster; repeating can lose namespace access to existing blocks
`start-dfs.sh`	Start HDFS daemons configured for the cluster	Requires correct config and SSH access for scripts; verify daemons and HDFS report
`stop-dfs.sh`	Request clean stop of HDFS daemons	Avoid killing processes as a substitute for clean stop
`start-yarn.sh`	Start ResourceManager/ NodeManager according to config	TP1 pseudo-distributed theory/practice; not a TP2 command
`stop-yarn.sh`	Stop YARN daemons	Not applicable to TP2's HDFS-only containers
`jps`	List Java processes for current user	A process can exist but fail to register; use `dfsadmin-report` too

TP2's container-specific workflow is `dockercomposeup-d...`, then `dockercomposeps`, `dockerlogs`, and `dockerexec...`. Do not assume host `start-dfs.sh` controls the containerized daemons.

8.2 HDFS filesystem shell: hdfs dfs

`hdfsdfs` is the filesystem shell. It operates on the filesystem selected by the configuration/URI. A local Linux command such as `ls` is not an HDFS command.

Table 8.2: HDFS filesystem shell: hdfs dfs reference table

HDFS filesystem shell: hdfs dfs — reference table		
Command	Syntax and example	Purpose / common mistake
List	`hdfsdfs-ls/user`	List HDFS path; empty or missing path differs from local filesystem
Create directory	`hdfsdfs-mkdir-p/tp2/blocks`	`-p` creates parents; need HDFS write permission
Upload	`hdfsdfs-putlocal.txt/user/name/`	Copies local file into HDFS; local path is on the client machine/container
Download	`hdfsdfs-get/user/name/a.txt./a.txt`	Copies from HDFS to local destination
Display	`hdfsdfs-cat/path/file`	Writes text/bytes to terminal; unsuitable for huge/binary files
Remove file	`hdfsdfs-rm/path/file`	Removes HDFS namespace entry; `-r` for directory tree; deletion is not a backup
Remove empty directory	`hdfsdfs-rmdir/path/empty`	Directory must be empty; use `-rm-r` only when intentionally recursive
Disk usage	`hdfsdfs-du-h/path`	Reports logical/space usage (flags/output depend on Hadoop version); replication affects physical storage
Permissions	`hdfsdfs-chmod755/path`	Changes HDFS permission bits; not Linux host file permissions or Docker ownership
Set replication	`hdfsdfs-setrep-w3/path/file`	Change target for an existing file and optionally wait; requires sufficient live DNs/capacity
Statistics	`hdfsdfs-stat'size=%brepl=%rblock=%o'/path/file`	TP2 uses `%b`, `%r`, `%o` for file size, replication and block size; consult version help if format differs

`-put` and `-copyFromLocal` are equivalent in typical use; `-get` and `-copyToLocal` likewise.

8.3 Administration and diagnostic commands

Table 8.3: Administration and diagnostic commands reference table

Administration and diagnostic commands — reference table		
Command	Example	Purpose
`hdfsdfsadmin-report`	`dockerexectp2-namenodehdfsdfsadmin-report`	Cluster capacity and live/dead DataNodes; TP2 expects Live datanodes (2), then (3)
`hdfsfsck`	`hdfsfsck/tp2/blocks/20MiB.bin-files-blocks-locations`	Inspect file/block health and locations; use to verify actual placement
`hdfsgetconf`	`hdfsgetconf-confKeydfs.blocksize`	Read a configured value as seen by this Hadoop client
`hdfsdfsadmin`	`hdfsdfsadmin-report`	Administrative/cluster operations, not ordinary path listing/upload

Difference: `hdfsdfs` manipulates filesystem paths and files. `hdfsdfsadmin` queries/administers the

HDFS cluster. `hdfsfsck` diagnoses filesystem/block health; it does not repair every issue merely because it is called.

8.4 Linux, Docker and Hadoop command families

Table 8.4: Linux, Docker and Hadoop command families reference table

Linux, Docker and Hadoop command families — reference table		
Family	Examples	What it addresses
Linux host	`pwd`, `ls`, `cat`, `mkdir`, `chown`, `df-h`	Ubuntu VM's files, processes, disks and ownership
HDFS filesystem	`hdfsdfs-ls`, `-put`, `-get`	HDFS namespace/data through Hadoop client
Hadoop admin	`hdfsdfsadmin-report`, `hdfsfsck`, `hdfsgetconf`	HDFS cluster status/configuration/diagnostics
Docker	`dockerps`, `dockerlogs`, `dockerexec`, `dockercomposeup`	Containers, images, networks, mounts and lifecycle

Example: `lsdata/datanode1` inspects the Ubuntu bind-mount directory. `hdfsdfs-ls/` lists HDFS. `dockerps` lists containers. They are not interchangeable.

8.5 TP1 quick validation

```
hadoop version
hdfs dfs -ls /
echo "Bonjour Hadoop" > test.txt
hdfs dfs -put test.txt /user/hadoop/
hdfs dfs -cat /user/hadoop/test.txt
```

If the HDFS directory is not present, create it with `hdfsdfs-mkdir-p/user/hadoop`. Check `fs.defaultFS` and service status before assuming an upload problem.

8.6 Additional requested filesystem commands

Table 8.5: Additional requested filesystem commands reference table

Additional requested filesystem commands — reference table		
Command	Syntax and example	Expected behavior and common mistakes
Capacity	`hdfsdfs-df-h/`	Shows HDFS capacity and available space. It is not the Ubuntu disk report (`df-h/` on Linux).
Change owner	`hdfsdfs-chownuser:group/path`	Changes HDFS ownership and generally requires HDFS superuser privileges. It does not change Linux bind-mount ownership.

Additional requested filesystem commands — continued		
Recursive delete	`hdfsdfs-rm-r/path/directory`	Removes an HDFS directory tree. Verify the path; this is destructive and is not a Linux local deletion.

HDFS `-chmod` and `-chown` change HDFS metadata. TP2's Linux `chown-R1001:100...` applies to bind-mounted storage so the container Hadoop user can write; these are separate permission layers.

Docker and Hadoop Deployment

9.1 Docker concepts

Table 9.1: Docker concepts reference table

Docker concepts — reference table	
Term	Definition
Docker Engine	Host service/runtime that builds, pulls, creates and runs containers
Image	Versioned, mostly immutable template of filesystem and metadata used to create containers
Container	A running or stopped instance of an image with isolated processes, network and mounts
Docker Compose	YAML-based way to define and manage related services, networks, mounts and profiles
Service	A Compose-described workload, such as `namenode` or `datanode1`
Network	Virtual connectivity among containers; Compose bridge provides service DNS
Bind mount	Maps a specific host directory/file into a container path
Named volume	Docker-managed persistent storage object not directly identified as a host project path

Containers share the host/VM kernel; a virtual machine runs its own guest OS kernel. Containers are not miniature independent physical servers.

9.2 Four layers in TP2

```
Physical computer (hardware)
  +-- VirtualBox
       +-- Ubuntu VM (one Linux host, Docker Engine, persistent project folder)
            +-- Compose bridge network: hadoop-net
            +-- NameNode container (hostname/name: namenode)
            +-- DataNode 1 container (hostname: datanode1)
            +-- DataNode 2 container (hostname: datanode2)
                [optional profile: DataNode 3]
```

Three Hadoop containers inside one VM are **three logical services**, not three independent physical machines. A VM, host disk, host kernel or power failure may affect all of them. Therefore, a three-container experiment teaches distribution and HDFS replica placement, but does not demonstrate resilience to loss of the physical computer.

9.3 Compose, networking, ports and names

- `image:apache/hadoop:3.5.0`: same Hadoop 3.5.0 image/Java 17 base for the services.
- `hostname` and service DNS: containers on `hadoop-net` can resolve `namenode`, `datanode1`, etc., without fixed IPs.
- `depends_on`: controls startup ordering, not application readiness; a DataNode may still need retries while NameNode initializes.
- `restart:unless-stopped`: Docker restarts a container after certain failures/restarts unless explicitly stopped.
- `profiles:["extension"]`: `datanode3` is opt-in via `dockercompose--profileextension...`.
- `dockercomposeconfig`: parse/normalize Compose config; catches YAML/model errors but not every runtime issue.

Table 9.2: Compose, networking, ports and names reference table

Compose, networking, ports and names — reference table		
Port	TP2 role	Access
8020	NameNode RPC for Hadoop clients/DataNodes	Internal Docker network in TP2; not published to Ubuntu
9870	NameNode HTTP web interface	Published to `127.0.0.1:9870` on Ubuntu VM
9866	DataNode data-transfer listener	HDFS data path; not published in TP2 Compose
9864	DataNode HTTP endpoint	DataNode web/HTTP; not published in TP2 Compose
9000	NameNode endpoint in TP1 pseudo-distributed example	Do not confuse with TP2's 8020
8088	ResourceManager web UI in TP1 YARN example	Not used in TP2

`ports:"127.0.0.1:9870:9870"` has form host-address:host-port:container-port. Changing host port to 19870 would allow `http://localhost:19870` on the VM while the NameNode still listens internally on 9870. TP2 asks to test this temporarily, validate with `dockercomposeconfig`, then restore.

9.4 Bind mounts and persistence

TP2 maps project directories, for example:

```
volumes:
  - ./data/namenode:/data/hdfs/name
```

Left side: Ubuntu project path (`./data/namenode`). Right side: path seen by the container (`/data/hdfs/name`). The NameNode property must point to the **container** path. Distinct host paths for each DataNode keep their data directories separate even though the internal path can be the same.

`dockercomposedown` removes the containers and Compose network but does not delete ordinary bind-mounted project directories. Therefore HDFS files can survive container recreation. Do not add `-v` or

manually delete `data/` when data must be retained. Persistence still depends on the VM disk and correct mounts.

9.5 Permissions

In the TP2 image, the `hadoop` account uses UID 1001 and primary GID 100. Directories created by the Ubuntu user may not be writable by that container user. The Compose `initialisation` service runs as root only to execute:

```
chown -R 1001:100 /data/hdfs/name /data/hdfs/dn1 /data/hdfs/dn2 /data/hdfs/dn3
```

Then NameNode/DataNodes run as the non-root Hadoop user. This is a host/container UID/GID mapping issue, not HDFS file permission `chmod`. Do not recursively change the project ownership afterward in a way that removes container write access.

9.6 TP1 optional Docker approach: BDE2020 image

This Compose example is distinct from the manual Hadoop 3.3.6 installation and from TP2's Apache Hadoop 3.5.0 image. TP1 uses BDE2020 Hadoop 3.1.3 with Java 8.

```
version: '3'
services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.1.3-java8
    container_name: namenode
    ports:
      - "9870:9870"
      - "9000:9000"
    environment:
      - CLUSTER_NAME=hadoop-cluster
    volumes:
      - hadoop_namenode:/hadoop/dfs/name
  datanode:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.1.3-java8
    container_name: datanode
    ports:
      - "9864:9864"
    environment:
      - SERVICE_PRECONDITION=namenode:9870
    volumes:
      - hadoop_datanode:/hadoop/dfs/data
volumes:
  hadoop_namenode:
  hadoop_datanode:
```

Start with `docker-composeup-d`, inspect `dockerlogsnamenode`, and open `http://localhost:9870`. TP1 also mentions Cloudera/Hortonworks sandbox VMs as optional preconfigured environments; they are not TP2.

Advanced TP2 Configuration

Figure 8 shows the TP2 network; Figure 7 shows the host/VM/container layers. TP2 is HDFS-only.

10.1 TP2 operation-by-operation logic

Each row gives the objective, underlying reason, expected proof and a safe diagnostic if it fails.

Table 10.1: TP2 operation-by-operation logic reference table

TP2 operation-by-operation logic — reference table

Operation	Why / internal logic	Expected result and checks
Prepare Ubuntu VM	One controlled Linux host for Docker Engine; inspect CPU, RAM, disk and architecture	`cat/etc/os-release`, `free-h`, `nproc`, `df-h/`; enough resources for the lab
Verify Docker/ Compose	Confirm runtime and permission before changing files	`docker--version`, `dockercomposeversion`, `dockerps`, `sudosystemctlis-activedocker`
Pull `apache/hadoop: 3.5.0`	Download exact versioned image once	`dockerrun--rm--entrypointhadoopapache/hadoop:3.5.0version`; should report 3.5.0
Check Java 17	Confirm Java packaged in image, not host Ubuntu	`dockerrun--rm--entrypointjavaapache/hadoop:3.5.0-version`; no need to install Java on Ubuntu for these services
Create project/config/ data dirs	Separate config from persistent NameNode/DN storage	`mkdir-p...`; inspect `pwd` and folder names
Copy original XML/ env from image	Understand defaults before deciding changes	Temporary stopped container + `dockercp`; remove only temporary container
Inspect `hadoop-env.s h`	Check image Java path and launch environment	`grep-nJAVA_HOME...`; TP2 says retain working Java 17 setup
Configure `core-site. xml`	Tell Hadoop clients default NameNode URI	`fs.defaultFS=hdfs://namenode:8020`; verify XML and loaded value
Configure `hdfs-site. xml`	Set block/replication/ storage/listener parameters	`getconf` values; paths must agree with mounts
Define Compose services	Start one NameNode and two DNs with consistent image/ config	`dockercomposeconfig`; correct service names, mounts and network
Inspect persistent dirs	Ensure data will live outside disposable container layer	`dockerinspect...--format'{{json.Mounts}}'`; host Source maps to container Destination
Run permission initializer	Make directories writable for UID:GID 1001:100	`dockercomposerun--rm--no-depsinitialisation`; inspect `ls-lddata/...`
Format NameNode once	Create empty HDFS namespace metadata in persistent NameNode path	Run before first cluster start; inspect `data/namenode/current`; never repeat for existing data
Start NameNode then DataNodes	Start the master metadata service, then workers	`dockercomposeup-d...`; inspect `dockercomposeps` and logs

TP2 operation-by-operation logic — continued		
Check live DNs	Container Running is not equivalent to HDFS registered	`hdfsdfsadmin-report` should show 2 live DataNodes
Open web UI	Observe NameNode status through HTTP	From Ubuntu VM: `http://localhost:9870`; expected two live DNs
Check loaded config	Confirm the config actually read by the process/client	`hdfsgetconf-confKey...`; initial values include 2 and 8388608
Inspect bridge/DNS	Avoid hard-coding changing IPs	`dockernetworkinspect...`, `getenthostsnamenode`
Check ports	Distinguish RPC, HTTP and host publication	Container can reach `namenode:8020`; only 9870 is published to VM loopback
Upload 20 MiB file	Create known input to inspect blocks	`dd...bs=1Mcount=20`, then `hdfsdfs-put`; expected three logical blocks
Inspect blocks	Verify actual block IDs/ locations and replication	`hdfsfsck...-files-blocks-locations`; expect 3 blocks, 2 replicas each once converged
Start third DN	Opt into extension without altering initial topology	`dockercompose--profileextensionup-ddatanode3`; report should reach 3 live DNs
`-setrep-w3` existing file	Change this file's target replication and wait for convergence	`-stat` and `fsck`; wait may stall if third DN is absent or capacity unavailable
Set XML default to 3	Default applies to newly created files	Back up XML; edit only replication property; recreate mounted services; `getconf` should show 3
Recreate containers	Demonstrate mounts preserve namespace/blocks	`composedown`, then `up`; HDFS listing/report still works, no reformat
Restart VM	Test Docker startup/restart policy and data persistence	After boot, check `dockerps`, HDFS report, web UI; start Compose if needed
Diagnose wrong NameNode address	Separate a client endpoint/ DNS error from filesystem format	`getconf`, `getenthosts`, correct URI; use `hdfs dfs-fshdfs://namenode:8020-ls/`; do not format

10.2 dfs.replication versus hdfs dfs -setrep

Table 10.2: dfs.replication versus hdfs dfs -setrep reference table

dfs.replication versus hdfs dfs -setrep — reference table		
Method	Affects	Example
`dfs.replication` in `hdfs-site.xml`	Default target for **newly created** files using that config	TP2 changes 2 to 3, recreates services, then uploads `test-rep3.txt`
`hdfsdfs-setrep[-w]Nfile`	Target replication of a specified **existing file**	TP2 changes existing `20MiB.bin` from 2 to 3

Changing the config does not automatically rewrite metadata/replica count for existing files. `-setrep` asks the NameNode to adjust replicas. Convergence depends on live DataNodes, capacity and scheduling; verify rather than assuming.

10.3 Diagnose a wrong NameNode address

In TP2, `namenode-inexistant` is a deliberate bad DNS name. A client command using that authority fails even if the actual NameNode is healthy, because the client has been directed to the wrong host. Check `hdfsgetconf-confKeyfs.defaultFS`, `getenthostsnamenode`, and `dockernetworkinspect`. Correct the endpoint in the command or mounted XML and recreate/restart the relevant process if needed. Formatting is unrelated and dangerous.

10.4 Compose structure: compact TP2-aligned example

The following excerpt highlights the architecture and lifecycle mechanisms from TP2. The assignment's full file also specifies the same image, entrypoint, environment, restart policy and XML mounts for each DataNode; `datanode3` is enabled only with the `extension` profile.

```
name: tp2-hadoop

services:
  initialisation:
    image: apache/hadoop:3.5.0
    profiles: ["setup"]
    user: "0:0"
    entrypoint: ["/bin/sh", "-c"]
    command:
      - "chown -R 1001:100 /data/hdfs/name /data/hdfs/dn1 /data/hdfs/dn2 /data/hdfs/dn3"
    volumes:
      - ./data/namenode:/data/hdfs/name
      - ./data/datanode1:/data/hdfs/dn1
      - ./data/datanode2:/data/hdfs/dn2
      - ./data/datanode3:/data/hdfs/dn3

  namenode:
    image: apache/hadoop:3.5.0
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports:
      - "127.0.0.1:9870:9870"
    volumes:
      - ./data/namenode:/data/hdfs/name
      - ./config/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
      - ./config/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
    networks: [hadoop-net]
    restart: unless-stopped

  datanode1:
    image: apache/hadoop:3.5.0
    hostname: datanode1
    command: ["hdfs", "datanode"]
    depends_on: [namenode]
    volumes:
      - ./data/datanode1:/data/hdfs/data
      - ./config/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
      - ./config/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
    networks: [hadoop-net]
    restart: unless-stopped
```

```
  datanode2:
    image: apache/hadoop:3.5.0
    hostname: datanode2
    command: ["hdfs", "datanode"]
    depends_on: [namenode]
    volumes:
      - ./data/datanode2:/data/hdfs/data
      - ./config/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
      - ./config/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
    networks: [hadoop-net]
    restart: unless-stopped

  datanode3:
    image: apache/hadoop:3.5.0
    profiles: ["extension"]
    hostname: datanode3
    command: ["hdfs", "datanode"]
    depends_on: [namenode]
    volumes:
      - ./data/datanode3:/data/hdfs/data
      - ./config/core-site.xml:/opt/hadoop/etc/hadoop/core-site.xml:ro
      - ./config/hdfs-site.xml:/opt/hadoop/etc/hadoop/hdfs-site.xml:ro
    networks: [hadoop-net]
    restart: unless-stopped

networks:
  hadoop-net:
    driver: bridge
```

In the complete TP2 file, the NameNode and each DataNode also set `HADOOP_CONF_DIR=/opt/hadoop/etc/hadoop` and use the image's `dumb-init` entrypoint. The writable bind mounts are data paths; XML mounts are read-only. The Compose hostname/service names support internal DNS; they do not imply separate physical hosts.

10.5 Representative TP2 command sequence

Run from `~/bigdata/tp2-hadoop` after reviewing the provided configuration. The format command is for a **new, empty NameNode namespace only**:

```
docker pull apache/hadoop:3.5.0
docker run --rm --entrypoint java apache/hadoop:3.5.0 -version
docker run --rm --entrypoint hadoop apache/hadoop:3.5.0 version
docker compose config
docker compose run --rm --no-deps initialisation

# Only before the first start of a new, empty HDFS namespace:
docker compose run --rm --no-deps --entrypoint hdfs namenode namenode -format -nonInteractive

docker compose up -d namenode
docker compose up -d datanode1 datanode2
docker exec tp2-namenode hdfs dfsadmin -report
docker exec tp2-namenode hdfs getconf -confKey fs.defaultFS
docker compose --profile extension up -d datanode3
```

```
docker exec tp2-namenode hdfs dfsadmin -report
```

The precise container names depend on the complete Compose file; TP2 names them `tp2-namenode`, `tp2-datanode1`, etc. Confirm `dockercomposeps` and the assignment file before using `dockerexec`.

Big Data Applications

Hadoop components can store and prepare large datasets for downstream analysis. Storing data or calculating aggregates is not the same thing as training a machine-learning model. A model may be trained by a separate engine using a prepared dataset.

Table 11.1: Course reference table

Course — reference table			
Application	Problem and data	Why distributed Big Data helps	Possible Hadoop role
Log analysis	Search and summarize web/server logs at high volume	Retain and scan many days of events efficiently	HDFS stores raw logs; MapReduce/Spark aggregates errors, latency and usage
E-commerce recommendations	Combine clicks, searches, purchases and catalog data	Large event history and joins across products/users	HDFS/data lake stores events; Hive/Spark prepares features; a recommendation model ranks items
Financial fraud detection	Detect unusual payment patterns from transactions and device signals	Many events, relationships and evolving histories	Batch HDFS pipeline builds features and reports; a low-latency service may make real-time decisions
Telecom analytics	Understand call records, network counters, location and faults	High event rate and geographic/time aggregation	HDFS stores history; batch jobs find usage trends or congested cells
Predictive maintenance	Predict failures from machine telemetry and repair logs	Long sensor histories across many machines	Store/clean sensor records; aggregate features for a separately trained model
Industrial IoT	Monitor sensors, alarms and equipment states	Variety, velocity and scale across sites	Ingest and retain events; batch analytics detects trends; streaming platform handles immediate alerts
Healthcare analysis	Study de-identified records, claims, lab values or imaging metadata	Large longitudinal cohorts and diverse formats	Governed data lake supports cohort construction/ETL; privacy and access control are essential
Social media analysis	Measure topics, engagement and trends in posts/media	High-volume unstructured and semi-structured content	Distributed storage and text-processing jobs; NLP model provides classification/sentiment
Data lake / ETL	Ingest raw files, validate, transform and publish curated datasets	Many source formats and large historical data	HDFS stores raw and curated zones; Hive/Spark/MapReduce transform data; catalog/governance tracks it

11.1 What Hadoop does not do by itself

HDFS stores and YARN schedules; MapReduce processes a specified job. None automatically guarantees that business data are correct, that a fraud model is accurate, or that a medical conclusion is valid. Those require data quality, suitable algorithms, evaluation, privacy and domain expertise.

Complementary Theory

12.1 High Availability, checkpointing and Erasure Coding

- **High Availability (HA):** design that reduces service interruption after a component failure. HDFS HA commonly uses Active and Standby NameNodes, shared edit-log coordination (such as JournalNodes), and failover control (such as ZKFC). HA is distinct from merely having replicas of data.
- **Checkpointing:** periodically merge the NameNode's namespace image and edit log to create a newer checkpoint, limiting how much edit history must be replayed. A SecondaryNameNode/CheckpointNode is a checkpointing role, not a hot standby.
- **Erasure Coding (EC):** encode data into data and parity fragments distributed across nodes; can provide fault tolerance with less storage overhead than full replication, at the cost of more computation/network work during recovery or some access patterns. It is not the same as ordinary replication.
- **Data locality:** schedule computation near the data to reduce network transfer; a preference, not a guarantee that every task runs on the exact node holding a block.
- **Fault tolerance vs HA:** fault tolerance concerns correct service despite a defined fault; HA emphasizes minimizing service downtime. A system can have durable replicated blocks while the NameNode control plane is unavailable.

12.2 HDFS versus a traditional filesystem

HDFS distributes large files into blocks over multiple DataNodes and offers replication and high-throughput streaming. A traditional local filesystem manages files on one host's storage and often supports low-latency random updates and local POSIX operations. HDFS relaxes some filesystem semantics and has network/metadata behavior; it is not simply "a larger ext4."

12.3 Hadoop versus Hive

Hadoop is the broader project/core platform, including HDFS, YARN, MapReduce and Common. Hive is a higher-level SQL-like data warehouse/query system that can execute using distributed engines and access data in distributed storage. Hive is not a storage filesystem and is not one of Hadoop's four core modules.

12.4 Important conceptual distinctions

Table 12.1: Important conceptual distinctions reference table

Important conceptual distinctions — reference table	
Pair	Correct distinction
HDFS vs YARN	HDFS stores files/blocks; YARN manages compute resources
HDFS vs MapReduce	HDFS stores input/output; MapReduce processes data
NameNode vs DataNode	NameNode manages namespace/block metadata; DataNode stores/serves blocks
NameNode vs SecondaryNameNode	Primary online namespace service vs periodic checkpointing helper; SecondaryNameNode is not automatic failover
RM vs NM	Cluster-level resource authority/scheduler vs per-node agent managing containers
Image vs container	Template vs created/running instance
Host port vs container port	Published endpoint on VM vs service listener inside container
Linux file vs HDFS file	Data on Ubuntu filesystem vs namespace/data managed by HDFS
Replication vs block size	Copies per block vs maximum nominal size of a block
Replication vs backup	Live copies for fault tolerance vs independent recoverable historical copy

12.5 Complementary theory: databases, CAP, partitioning, and ecosystem

The following concepts are useful exam extensions; they are **not additional services deployed in TP2**.

12.5.1 SQL and NoSQL; relational systems and distributed storage

SQL commonly refers to a query language and relational database systems with schemas, joins and transactional guarantees. NoSQL is a broad family (key-value, document, wide-column, graph) with different data models and trade-offs; it does not mean “no queries” or “no schema.” Select systems by access pattern, consistency, transactions, scale and operations. A relational database is designed for structured records and transactions; HDFS is a distributed file store for large files and high-throughput scans, not a drop-in transactional database.

12.5.2 Database versus data warehouse

A database may support operational workloads (OLTP): frequent small reads/writes and transaction integrity. A data warehouse is organized for analytical queries (OLAP), historical integration and aggregation. A warehouse is an architectural role, not necessarily one product. Hadoop/HDFS can store raw and curated analytical data; Hive provides a SQL-like warehouse/query layer but is not HDFS.

12.5.3 CAP theorem

For a distributed data service experiencing a network partition, CAP describes a trade-off between **consistency** (clients observe a single up-to-date view) and **availability** (each request receives a response), while partition tolerance addresses continued operation despite communication splits. It is not accurate

to say a system simply "chooses any two" under all circumstances: a partition is a failure condition, and the design must decide which behavior to preserve during it. CAP consistency is not ACID consistency.

12.5.4 Partitioning versus replication

Partitioning (sharding) divides a dataset among nodes to distribute capacity and work. Replication stores copies of the same data on multiple nodes to improve availability/durability. Systems often combine both: partition first, then replicate each partition. HDFS blocks partition file data; its replication factor controls copies of each block.

12.5.5 Hadoop ecosystem roles

Table 12.2: Hadoop ecosystem roles reference table

Hadoop ecosystem roles — reference table		
Project	Typical role	Relationship to core Hadoop
Hive	SQL-like warehouse and query layer	Works over distributed data; not a core HDFS/ YARN module
Pig	Data-flow scripting for batch transformations	Higher-level programming layer; legacy/less common in new designs
HBase	Distributed wide-column NoSQL database	Can use HDFS; offers record-oriented access unlike HDFS alone
Sqoop	Historical bulk transfer between relational databases and Hadoop stores	Ingestion/export tool; use depends on release/ support status
Flume	Event/log ingestion into Hadoop-oriented stores	Collects events; not a MapReduce-equivalent processing engine
Oozie	Workflow/job coordination for Hadoop-era jobs	Orchestrates jobs; not the YARN scheduler
Spark	General distributed compute engine	May run with YARN and read HDFS; separate from MapReduce

12.5.6 Batch versus real-time processing

Batch processes a bounded dataset as a scheduled or submitted job; stream processing continuously handles events with low latency and state management. Hadoop MapReduce is a classic batch model. An architecture may retain events in HDFS for historical batch analysis while a separate stream engine handles immediate alerts.

Essential Definitions

1. **Big Data:** Data/workloads whose scale, arrival rate, diversity or operational demands call for scalable distributed approaches.
2. **Volume:** The amount of data generated or stored.
3. **Velocity:** The rate at which data arrive and must be processed.
4. **Variety:** Diversity of data formats, structures and sources.
5. **Veracity:** Reliability, completeness and uncertainty of data.
6. **Value:** Useful outcome obtained from data.
7. **Structured data:** Data organized under a regular, predefined schema, often tabular.
8. **Semi-structured data:** Data with tags/keys or partial organization but flexible records, such as JSON.
9. **Unstructured data:** Content not naturally represented by a fixed tabular schema, such as images or free text.
10. **Distributed computing:** Coordinated computation across networked machines.
11. **Parallel processing:** Simultaneous execution of parts of a workload.
12. **Horizontal scaling:** Increasing capacity by adding machines/workers.
13. **Vertical scaling:** Increasing resources on one machine.
14. **Fault tolerance:** Ability to continue or recover correctly under specified component failures.
15. **Availability:** Degree to which a service is ready for use.
16. **Hadoop:** Open-source framework/ecosystem for distributed storage and processing.
17. **Hadoop Common:** Shared libraries and utilities used by Hadoop modules.
18. **HDFS:** Hadoop's distributed filesystem for large, high-throughput datasets.
19. **NameNode:** HDFS service that manages namespace metadata and block mappings.
20. **DataNode:** HDFS worker service that stores and serves block replicas.
21. **HDFS block:** Logical unit into which an HDFS file is divided for storage.
22. **Block size:** Configured maximum nominal size of a full HDFS block.
23. **Replication factor:** Target number of copies of each file block.
24. **SecondaryNameNode:** HDFS checkpointing service; not a hot standby NameNode.
25. **YARN:** Hadoop resource-management and scheduling framework.
26. **ResourceManager:** Cluster-wide YARN authority that accepts applications and allocates resources.
27. **NodeManager:** Per-worker YARN agent that launches/monitors containers and reports resources.

28. **ApplicationMaster:** Per-application coordinator that requests resources and tracks that application's tasks.
29. **Container (YARN):** Allocation of resources in which an application task/process runs.
30. **MapReduce:** Distributed batch model with map, shuffle/sort and reduce stages.
31. **Shuffle:** Transfer, partitioning and grouping of intermediate mapper output by key for reducers.
32. **Data locality:** Preference to run computation near the data it processes.
33. **Batch processing:** Processing a bounded dataset as a job, often at scheduled intervals.
34. **Stream processing:** Continuously processing events as they arrive.
35. **Docker image:** Versioned template used to create containers.
36. **Docker container:** Isolated process environment created from an image.
37. **Docker Compose:** Tool and YAML model for defining and managing multi-service applications.
38. **Bind mount:** Mapping of a specific host filesystem path into a container path.
39. **NameNode formatting:** Initialization of a new HDFS namespace and its persistent metadata.
40. **High Availability:** Architecture intended to keep a service usable with minimal interruption during failures.
41. **Checkpointing:** Merging namespace image and edits to create a current metadata checkpoint.
42. **Erasure Coding:** Fault-tolerant encoding into data/parity fragments with lower storage overhead than full replication in suitable cases.
43. **Schema-on-read:** Interpret/validate structure when data are queried rather than requiring all structure at ingestion.
44. **Data lake:** Repository that stores data in varied formats for later governed use and analysis.
45. **HDFS replication:** Storage of multiple copies of a block on DataNodes to tolerate certain failures; not a backup.

Comparison Questions and Answers

Use the model answer as a compact written response; expand it with an example if the question asks "explain."

1. **Standalone vs pseudo-distributed:** Standalone runs locally without HDFS daemons; pseudo-distributed runs separate Hadoop daemons on one host and can use HDFS/YARN. Both share one physical failure domain.
2. **Pseudo-distributed vs fully distributed:** Both run distributed services; pseudo-distributed places them on one host for learning, while fully distributed uses multiple networked hosts for scale and host-level failure separation.
3. **NameNode vs DataNode:** NameNode stores namespace/block metadata and coordinates; DataNode stores actual blocks, serves clients and reports health.
4. **NameNode vs SecondaryNameNode:** NameNode is the live metadata authority; SecondaryNameNode checkpoints metadata periodically and does not automatically replace the NameNode.
5. **HDFS vs YARN:** HDFS handles distributed storage; YARN allocates cluster resources to applications.
6. **HDFS vs MapReduce:** HDFS persists input/output; MapReduce computes over records and writes results.
7. **ResourceManager vs NodeManager:** RM allocates at cluster scope; NM manages containers and reports resources on one worker.
8. **ApplicationMaster vs ResourceManager:** AM coordinates one application; RM coordinates cluster-wide resource allocation and application admission.
9. **Container vs virtual machine:** YARN container is a resource allocation/process environment; VM virtualizes a guest OS and kernel.
10. **core-site.xml vs hdfs-site.xml:** Common filesystem settings such as `fs.defaultFS` vs HDFS-specific storage/block/replication/listener settings.
11. **hdfs-site.xml vs hadoop-env.sh:** XML Hadoop properties vs shell environment/JVM launch configuration.
12. **mapred-site.xml vs yarn-site.xml:** MapReduce framework/job settings vs YARN/NodeManager settings.
13. **`hdfsdfs` vs `hdfsdfsadmin`:** Filesystem path operations vs HDFS administrative/cluster operations.
14. **`hdfsdfsadmin -report` vs `hdfsfsck`:** Cluster capacity/worker health vs filesystem/block health and locations.
15. **`dfs.replication` vs `-setrep`:** Default for newly created files vs target update for a selected existing file.
16. **Block size vs replication factor:** Bytes in a logical block vs copies of each block.

17. **Replication vs backup:** Active distributed copies aid availability/recovery from node loss; backup is separate, retained recovery data.
18. **HDFS vs local filesystem:** Distributed large-file storage across DataNodes vs host-local file management and semantics.
19. **Hadoop vs Hive:** Core distributed platform/ecosystem vs SQL-like warehouse/query layer using distributed storage/compute.
20. **Hadoop MapReduce vs Spark:** Classic disk-oriented batch framework vs separate general compute engine supporting in-memory and other workloads; Spark may use HDFS/YARN.
21. **Docker image vs container:** Reusable template vs instance made from it.
22. **Bind mount vs named volume:** Explicit host path mapped into container vs Docker-managed volume object.
23. **Host port vs container port:** External port on Ubuntu used to reach a service vs port on which the process listens inside its container.
24. **Docker bridge DNS vs fixed container IP:** Stable service-name resolution on the Compose network vs IP that can change after recreation.
25. **Container recreation vs data deletion:** Replacing containers need not remove bind-mounted data; deleting the host data directory does.
26. **Three containers vs three physical hosts:** Logical service separation only vs independent hardware/OS failure domains.
27. **Batch vs stream:** Bounded dataset/job with delayed result vs unbounded events processed continuously with low latency.
28. **Vertical vs horizontal scaling:** Add resources to one host vs add hosts/workers.
29. **Fault tolerance vs high availability:** Correctness/recovery despite a fault vs minimizing service interruption; related but not identical goals.
30. **Checkpointing vs HA:** Metadata log/image compaction and recovery support vs active/standby service architecture with failover.

Theoretical Examination Questions

These are practice questions, not predictions. Each model answer states why it is correct and lists key terms to include.

15.1 Fundamentals and Hadoop

Q1. Define Big Data and explain why it has no universal size threshold. Answer: Big Data is data/workloads whose volume, velocity, variety, veracity or required analysis exceeds the practical capabilities of conventional tools. The threshold is relative to available systems, latency and use case, not a universal byte count. **Why:** a dataset manageable on one system today may not be manageable under a tighter latency or quality requirement. **Key terms:** 5 Vs, scale, distributed processing, context.

Q2. Name and explain the 5 Vs. Answer: Volume is amount, velocity is arrival/processing rate, variety is format diversity, veracity is reliability/uncertainty, and value is useful outcome. **Why:** the framework captures distinct engineering and business challenges. **Key terms:** volume, velocity, variety, veracity, value.

Q3. Distinguish structured, semi-structured and unstructured data. Answer: Structured data follow a fixed schema (table); semi-structured data use keys/tags with flexible fields (JSON/XML); unstructured content lacks a natural fixed table (image/audio/free text). **Why:** the distinction concerns organization, not whether data contain any internal structure. **Key terms:** schema, table, flexible, content.

Q4. Explain horizontal and vertical scaling. Answer: Vertical scaling increases resources of one machine; horizontal scaling adds machines/workers and partitions work. **Why:** distributed Big Data systems are designed to scale out, though coordination adds complexity. **Key terms:** scale-up, scale-out, nodes, partition.

Q5. State the four core Hadoop modules and their roles. Answer: Common provides shared utilities; HDFS stores distributed files; YARN manages cluster resources; MapReduce processes batch data. **Why:** these roles distinguish infrastructure from computation. **Key terms:** Common, HDFS, YARN, MapReduce.

Q6. Why does Hadoop use distributed storage and computation? Answer: Partitioning permits aggregate storage/throughput beyond one machine, while distributed tasks process partitions in parallel and can recover from some component failures. **Why:** commodity nodes can fail, so coordination and recovery are central. **Key terms:** partitioning, parallelism, scale-out, fault tolerance.

15.2 HDFS

Q7. What is the NameNode's role? Answer: It manages the HDFS namespace and metadata, including file-to-block and block-to-DataNode mappings, and coordinates client operations. It normally does not relay file bytes. **Why:** data are transferred directly between client and DataNodes. **Key terms:** namespace, metadata, block mapping, coordination.

Q8. What is the DataNode's role? Answer: It stores block replicas, serves client reads/writes,

performs block operations as directed, and reports health/blocks to the NameNode. **Why:** DataNodes hold the file contents, not the namespace authority. **Key terms:** blocks, replicas, heartbeat, block report.

Q9. Explain why HDFS divides files into blocks. Answer: Blocks allow a file to be distributed among nodes, replicated independently and processed in parallel. **Why:** file size is not limited to one machine's disk and workers can operate near partitions. **Key terms:** partition, block, distribution, parallel processing.

Q10. Calculate blocks and copies for a 20 MiB file, block size 8 MiB, replication 2. Answer: Three logical blocks: 8 + 8 + 4 MiB. There are six expected block copies after replication converges. **Why:** `ceil(20/8)=3`, then multiply by two. **Key terms:** logical blocks, final partial block, replication.

Q11. Does HDFS fill a final partial block to the configured size? Explain. Answer: No. Block size is a maximum nominal size; the final block stores only remaining bytes. **Why:** a 20 MiB file with 8 MiB blocks ends with a 4 MiB block. **Key terms:** block size, partial final block, no padding.

Q12. Describe an HDFS read. Answer: The client asks the NameNode for file metadata and block locations, then reads each block directly from an appropriate DataNode and verifies checksums. **Why:** the NameNode supplies control/metadata while DataNodes supply bytes. **Key terms:** metadata request, block locations, direct read, checksum.

Q13. Describe an HDFS write. Answer: The client requests file creation and block targets from the NameNode, then streams each block through a DataNode pipeline; acknowledgements return and metadata is finalized on close. **Why:** block bytes bypass the NameNode. **Key terms:** create, target selection, pipeline, acknowledgement, close.

Q14. What happens when a DataNode fails? Answer: Missing heartbeats lead the NameNode to mark it unavailable. Clients can use other replicas; HDFS schedules replacement replicas for under-replicated blocks if data and live capacity permit. **Why:** replication provides alternate copies. **Key terms:** heartbeat, live/dead, under-replication, re-replication.

Q15. What is the SecondaryNameNode, and what is it not? Answer: It periodically checkpoints namespace metadata by merging the filesystem image and edits. It is not a hot standby or automatic NameNode replacement. **Why:** checkpointing reduces recovery/replay work but does not provide active failover. **Key terms:** fsimage, edits, checkpoint, not HA.

Q16. Why is formatting the NameNode dangerous after first initialization? Answer: Formatting initializes a new namespace/cluster identity and metadata store; existing blocks may no longer correspond to that namespace. **Why:** it is not a harmless service restart. **Key terms:** namespace, metadata directory, one-time, data loss risk.

Q17. Explain replication factor and its limitation. Answer: It is the target number of copies of each file block. It helps tolerate some DataNode failures but consumes storage and does not protect against every shared failure or accidental deletion. **Key terms:** copies per block, availability, capacity, not backup.

15.3 YARN and MapReduce

Q18. What problem does YARN solve? Answer: It separates cluster resource management from individual processing engines, allowing applications to request and use containers across workers. **Why:** multiple frameworks can share cluster resources. **Key terms:** ResourceManager, NodeManager, ApplicationMaster, container.

Q19. Distinguish ResourceManager and NodeManager. Answer: ResourceManager is cluster-wide and allocates resources; NodeManager runs per worker and launches/monitors containers and reports

local resources. **Why:** one is global authority, the other is a machine agent. **Key terms:** cluster-level, per-node, scheduler, containers.

Q20. What is an ApplicationMaster? Answer: A per-application coordinator that negotiates resources with YARN and tracks that application's work. **Why:** application-specific monitoring is distinct from global resource allocation. **Key terms:** per-application, negotiate, track, containers.

Q21. Describe MapReduce's stages. Answer: Input is split into records; map emits intermediate key/value pairs; shuffle partitions/transfers/groups by key; reduce aggregates values; output is written. **Why:** each stage transforms data toward a grouped result. **Key terms:** map, key/value, shuffle/sort, reduce, output.

Q22. Why is shuffle important and potentially expensive? Answer: It routes mapper output to the reducer responsible for each key, often across the network, and groups/sorts it. **Why:** this makes all values for a key available together but incurs network, disk and serialization costs. **Key terms:** partition, network transfer, grouping, sort.

Q23. How do HDFS, YARN and MapReduce work together? Answer: HDFS stores input/output, YARN allocates containers, and MapReduce runs map/shuffle/reduce tasks in those containers. **Why:** storage, resource scheduling and computation are separate responsibilities. **Key terms:** storage, allocation, tasks, output.

15.4 Modes and configuration

Q24. What is the difference between standalone and pseudo-distributed mode? Answer: Standalone runs locally without Hadoop daemons and usually uses the local filesystem; pseudo-distributed runs separate Hadoop daemons on one host and can use HDFS/YARN. **Why:** one is local execution, the other simulates a cluster's processes. **Key terms:** local mode, daemons, one host.

Q25. What distinguishes a fully distributed cluster? Answer: Services run across multiple networked machines, enabling scale and separation of some host failures, with greater network/security/operations needs. **Why:** containers or processes on one host are not independent hosts. **Key terms:** multiple machines, workers, network, failure domain.

Q26. What is `fs.defaultFS`? Answer: The URI of the default filesystem used when a path has no explicit URI. TP2 uses `hdfs://namenode:8020`; TP1 pseudo-distributed uses `hdfs://localhost:9000`. **Why:** scheme, host and port direct client operations. **Key terms:** URI, default filesystem, NameNode, TP-specific values.

Q27. Compare `core-site.xml` and `hdfs-site.xml`. Answer: `core-site.xml` contains common filesystem settings such as `fs.defaultFS`; `hdfs-site.xml` contains HDFS-specific storage, block, replication and listener properties. **Why:** they configure different scopes. **Key terms:** common, HDFS-specific, URI, blocks.

Q28. Compare `hadoop-env.sh` and XML configuration. Answer: `hadoop-env.sh` is shell syntax for process environment/JVM launch settings; XML files define named Hadoop properties. **Why:** changing a shell variable is not the same as changing a filesystem property. **Key terms:** shell, environment, JVM, XML property.

15.5 Docker and TP2

Q29. What is the difference between an image and a container? Answer: An image is a template; a container is an instance created from it, with its own process lifecycle and configured mounts/

network. **Why:** many containers can use the same image. **Key terms:** template, instance, lifecycle.

Q30. Why do Docker service names matter in TP2? Answer: Compose bridge DNS resolves stable service names such as `namenode`, so peers can use names instead of changing container IPs. **Why:** recreated containers may get different IP addresses. **Key terms:** bridge, internal DNS, service name.

Q31. Distinguish bind mount and container filesystem. Answer: A bind mount maps a specific Ubuntu path to a container path; container-layer files are otherwise tied to container lifecycle. **Why:** TP2's bind mounts retain NameNode metadata and DN blocks when containers are recreated. **Key terms:** host path, container path, persistence.

Q32. Why is a running DataNode container not enough proof of a healthy HDFS cluster? Answer: The process may not have registered or may be failing internally. Use `hdfsdfsadmin-report`, logs and the NameNode UI to verify HDFS-level health. **Key terms:** container state, registration, live DataNode, report.

Q33. Explain the difference between ports 8020 and 9870 in TP2. Answer: 8020 is the NameNode RPC endpoint for Hadoop clients/services; 9870 is the NameNode HTTP web interface. TP2 publishes 9870 to Ubuntu loopback, while RPC stays on the Docker network. **Key terms:** RPC, HTTP, internal bridge, host publish.

Q34. Why are three Hadoop containers in one VM not three independent physical machines? Answer: They share the VM, kernel, virtual disk and physical host; a shared failure can affect all. **Why:** logical process isolation is not hardware failure isolation. **Key terms:** one VM, shared failure domain, physical host.

Q35. Why does `dfs.replication=3` not automatically make old files have three copies? Answer: The property supplies a default for newly created files; existing file metadata keeps its target until changed with `hdfsdfs-setrep`. **Key terms:** default, new files, existing metadata, setrep.

Q36. Why should TP2 inspect `hdfsgetconf`? Answer: It verifies the effective property seen by a Hadoop client, complementing the source XML. **Why:** wrong mounts, config directories or stale processes can make the edited file irrelevant. **Key terms:** effective config, HADOOP_CONF_DIR, mount, recreate.

15.6 Operations and applications

Q37. Distinguish `hdfsdfs` and `hdfsdfsadmin`. Answer: `hdfsdfs` performs path/file operations; `hdfsdfsadmin` performs cluster-level administration/reporting. **Why:** `-ls` and `-put` target the namespace, while `-report` describes DataNodes/capacity. **Key terms:** filesystem shell, administration, report.

Q38. Explain data locality. Answer: A scheduler prefers placing computation near the HDFS data it will process. **Why:** it can reduce network traffic and improve throughput, but exact placement is not guaranteed. **Key terms:** compute near data, network cost, preference.

Q39. Explain why replication is not a backup. Answer: Replicas are part of the live HDFS namespace and may reflect deletion/corruption; a backup is separately retained for recovery. **Why:** copies address some hardware failures, not all logical or site failures. **Key terms:** live copy, independent retention, deletion propagation.

Q40. Compare Hadoop and Spark. Answer: Hadoop includes distributed storage/resource-management/processing modules; Spark is a separate compute engine that can run on YARN and read HDFS, often using memory across stages. **Why:** Spark is not HDFS and does not make the entire Hadoop project obsolete. **Key terms:** project vs engine, HDFS, YARN, in-memory.

Practical Exercises with Corrections

Work each problem before reading the correction. Commands are illustrative and should be run only in a lab.

16.1 Exercise 1 — Count blocks

Question: A file is 20 MiB and block size is 8 MiB. How many blocks and what is the size of the final block? **Correction:** `ceil(20/8)=3`; sizes are 8, 8 and 4 MiB. The last block is not padded.

16.2 Exercise 2 — Count replicas

Question: The file in Exercise 1 has replication factor 2. How many block replicas are expected after convergence? **Correction:** `3ÃŮ2=6` block copies total. This is not six logical blocks or necessarily six distinct DataNodes.

16.3 Exercise 3 — Interpret TP2 properties

Given:

```
<property><name>dfs.blocksize</name><value>8388608</value></property>
<property><name>dfs.replication</name><value>2</value></property>
```

Question: Explain each value. **Correction:** 8,388,608 bytes = 8 MiB nominal block size; target two replicas per block. A 20 MiB file uses three logical blocks and six expected copies.

16.4 Exercise 4 — Find wrong TP2 endpoint

Given `fs.defaultFS=hdfs://localhost:9000` inside a DataNode container. **Question:** Why is this likely wrong for TP2? **Correction:** `localhost` refers to that container, not the NameNode container; TP2 peers resolve `namenode` on the Compose network and use RPC port 8020. Correct to `hdfs://namenode:8020` and confirm with `hdfsgetconf`. Do not format.

16.5 Exercise 5 — Distinguish host and container paths

Given bind mount `./data/datanode1:/data/hdfs/data` and XML `dfs.datanode.data.dir=file:///data/hdfs/data`. **Question:** Where are blocks stored? **Correction:** Hadoop writes to `/data/hdfs/data` inside the container; this maps to `./data/datanode1` on Ubuntu.

16.6 Exercise 6 — Diagnose a duplicate property

`hdfs-site.xml` contains two `dfs.replication` properties, one 2 and one 3. **Question:** Is this safe? What should you do? **Correction:** No; duplicate properties can create ambiguity/unexpected effective config. Keep one intended property, validate XML and run `hdfsgetconf-confKeydfs.replication` after recreating/restarting processes that load it.

16.7 Exercise 7 — Understand -setrep

An existing file has replication 2. The XML default changes to 3. **Question:** What is its existing replication target now? **Correction:** It remains 2 until changed for that file using `hdfsdfs-setrep3/path/file`. The XML default governs new files. Confirm with `-stat/fsck`.

16.8 Exercise 8 — YARN versus HDFS

Question: A Compose file starts a NameNode and two DataNodes but has no ResourceManager. Which TP2 function can work, and which cannot be inferred? **Correction:** HDFS storage/commands can work. YARN scheduling/MapReduce execution cannot be claimed from those services; TP2 explicitly deploys HDFS only.

16.9 Exercise 9 — Fix malformed XML

Given:

```
<configuration>
  <property><name>dfs.replication</name><value>2</property>
</configuration>
```

Question: Identify the problem. **Correction:** The `<value>` tag is not closed before `</property>`. Use `<value>2</value>`. Validate the XML and inspect Hadoop logs for parse errors.

16.10 Exercise 10 — Read Compose port mapping

Given `127.0.0.1:19870:9870`. **Question:** Which URL should be used in Ubuntu, and which port is inside the container? **Correction:** `http://localhost:19870`; container service still listens on 9870. RPC and Hadoop's internal configuration are unchanged.

16.11 Exercise 11 — DataNode container running but report shows 0 live

Question: Name three checks. **Correction:** `dockerlogstp2-datanode1`, inspect the shared bridge and `getenthostsnamenode`, and verify matching XML/`fs.defaultFS` and data-directory permissions. `dockercomposeps` alone proves only container state.

16.12 Exercise 12 — Third DataNode is absent

Question: What is the likely Compose action and what verifies success? **Correction:** Start with `dockercompose--profileextensionup-ddatanode3`. Check its logs, then `hdfsdfsadmin-report` for three live DataNodes. A running container alone is insufficient.

16.13 Exercise 13 — Why does -setrep -w 3 wait?

Question: Give two plausible causes and safe diagnostics. **Correction:** Third DN has not registered, or nodes lack usable space/are unhealthy. Check `hdfsdfsadmin-report`, `dockerlogstp2-datanode3`, and `hdfsfsck...-locations`; do not reformat.

16.14 Exercise 14 — Persistence after compose down

Question: The containers are gone but files remain. Explain. **Correction:** The NameNode metadata and DataNode blocks were in bind-mounted Ubuntu directories, which `dockercomposedown` does not delete. Bring services back with the same mounts; do not reformat.

16.15 Exercise 15 — Spot a dangerous command

Question: A student sees an HDFS listing failure after a bad hostname and proposes `hdfsnamenode -format`. Should they do it? **Correction:** No. Diagnose URI/DNS with `hdfsgetconf`, `getenthosts`, Compose network inspection and logs. Format affects namespace metadata, not address resolution, and can make stored files inaccessible.

Chapter 17

Troubleshooting Guide

Use a non-destructive diagnostic sequence: identify the layer (VM, Docker, process, network, Hadoop config, HDFS metadata), inspect logs/status, verify effective settings, then correct the narrow cause. Do not format or delete data as a generic first step.

Table 17.1: Course reference table

Course — reference table			
Symptom	Likely cause	Diagnostics	Corrective action
NameNode does not start	Invalid XML, missing/unwritable metadata path, unformatted new namespace, port conflict	`dockercomposeps`; `dockerlogs--tail80tp2-namenode`; inspect XML/mounts and `ls -lddata/namenode`	Fix XML/path/ownership. Format only if this is genuinely a new empty cluster and the intended metadata path is empty
DataNode cannot connect to NameNode	Wrong `fs.defaultFS`, DNS failure, network mismatch, NameNode not ready	`dockerlogstp2-datanode1`; `getenthostsnamenode`; `dockernetworkinspect...`; test RPC 8020	Correct the service URI/network and restart affected service; keep RPC internal to bridge if peers use it there
Docker says permission denied	User lacks access to Docker socket or Docker service is stopped	`dockerps`; `sudosystemctlstatusdocker`; inspect group/session	Start Docker if stopped; follow host's Docker access policy. Do not change HDFS ownership blindly
Port 9870 already occupied	Another service/cluster already publishes host port	`dockerps--format..`; inspect Compose port mappings	Stop the other intended cluster cleanly or temporarily map a different host port such as 19870, then restore if required
Wrong `fs.defaultFS`	Stale or incorrect XML, container localhost used instead of `namenode`	`hdfsgetconf-confKeyfs.defaultFS`; `getenthostsnamenode`; inspect mounted `core-site.xml`	Set the correct URI, e.g. TP2 `hdfs://namenode:8020`; recreate/restart client/service as needed
Invalid XML	Missing closing tag, typo, duplicate property or malformed file	`catconfig/*.xml`; check NameNode/DataNode logs; `dockercomposeconfig` does not validate Hadoop XML semantics	Fix well-formed XML and property spelling; keep one property definition; recreate affected containers
Missing HDFS data directory / permission denied	Mount path mismatch or directory not writable by image user	`dockerinspect...Mounts`; `ls-lddata/...`; logs; verify UID:GID 1001:100	Run the Compose `initialisation` service for the specified data paths; correct mount/XML path agreement
NameNode reports not formatted	New cluster's metadata directory is empty	Inspect logs and `data/namenode/current`	On a truly new lab namespace only, perform the prescribed one-time format; never format an existing cluster to fix unrelated errors

Course — continued			
`Livedatanodes(0)` or `(1)`	DNs failed registration, config or permissions issue, startup still in progress	`dockercomposeps`; both DN logs; `dfsadmin-report`; DNS and mounts	Fix underlying issue and allow registration; do not treat Running state as HDFS health
`getconf` value did not change	Wrong file mounted, duplicate property, wrong `HADOOP_CONF_DIR`, process/client not recreated	`dockerinspect` mounts; `echo$HADOOP_CONF_DIR`; `hdfsgetconf-confKey...`; logs	Correct source/mount, remove duplicate, recreate/restart relevant containers; no format needed
Third DataNode does not register	Compose profile not enabled, bad permissions/network/config	`dockercompose--profileextensionps`; `dockerlogstp2-datanode3`; `dfsadmin-report`	Start the `extension` profile service and correct its mounts/permissions/network
`-setrep-w3` appears stuck	Requested replicas cannot yet be placed; third DN down or insufficient capacity	Check live DNs and `fsck-locations`; inspect logs/capacity	Restore enough live storage nodes/capacity, then recheck; interrupting wait does not fix placement
HDFS data disappears after container recreation	Data was in container layer, mount omitted/changed, or wrong project directory used	`dockerinspect...Mounts`; inspect host `data/`; verify current working directory	Restore correct bind mounts and project directory. If namespace metadata was lost, preserve remaining disks and investigate; do not overwrite with format
NameNode web UI inaccessible	Wrong host port/bind address, port collision, service not listening	`dockercomposeps`; `dockerlogs`; inspect `ports`; browse on the Ubuntu VM	Use mapped URL (e.g. `http://localhost:9870`); resolve port conflict or restore correct mapping
Client uses `namenode-inexistant`	Deliberate/accidental invalid Docker DNS name	`getenthostsnamenode`; `hdfsgetconf`; inspect `-fs` argument	Use `namenode` or correct service name; no data or format change is involved

Multiple-Choice Questions

Choose one answer in each item. The correct letters vary. Each answer includes a short explanation.

1. Which is a core Hadoop module?
   A. Docker Compose B. HDFS C. VirtualBox D. PostgreSQL
   Answer: B. HDFS is a Hadoop core module.

1. Which V describes the diversity of JSON, images and tables?
   A. Value B. Velocity C. Variety D. Veracity
   Answer: C. Variety concerns formats and sources.

1. What does the TP2 NameNode primarily manage?
   A. Namespace and block metadata B. The Ubuntu kernel C. Map task code D. Docker image pulls
   Answer: A. The NameNode manages HDFS metadata.

1. Which component stores HDFS file blocks?
   A. ResourceManager B. NameNode C. ApplicationMaster D. DataNode
   Answer: D. DataNodes store/serve block replicas.

1. In TP2, what is `dfs.blocksize=8388608`?
   A. 8 bytes B. 8 MiB C. 8 GiB D. 8 replicas
   Answer: B. 8,388,608 bytes equals 8 MiB.

1. A 20 MiB file with 8 MiB blocks has how many logical blocks?
   A. 2 B. 4 C. 3 D. 5
   Answer: C. `ceil(20/8)=3`.

1. With three blocks and replication factor 2, the expected block-copy count is:
   A. 3 B. 5 C. 8 D. 6
   Answer: D. Three blocks times two copies.

1. Which port is the TP2 NameNode RPC port?
   A. 8020 B. 9870 C. 8088 D. 9864
   Answer: A. TP2 uses RPC 8020.

1. Which port provides the TP2 NameNode web UI?
   A. 9866 B. 9870 C. 8020 D. 9000
   Answer: B. The web UI is HTTP 9870.

1. Which property selects the default filesystem URI?
   A. `dfs.replication` B. `yarn.nodemanager.aux-services` C. `fs.defaultFS` D. `dfs.blocksize`
   Answer: C. It supplies the default filesystem URI.

1. Where is `fs.defaultFS` normally configured?
   A. `core-site.xml` B. `mapred-site.xml` C. `hadoop-env.sh` D. `compose.yaml`
   Answer: A. It is a common filesystem setting.

1. Which file configures `mapreduce.framework.name=yarn`?
   A. `core-site.xml` B. `yarn-site.xml` C. `hdfs-site.xml` D. `mapred-site.xml`
   Answer: D. This is a MapReduce framework property.

1. What does `mapreduce_shuffle` enable?
   A. NameNode checkpointing B. MapReduce shuffle service at NodeManager C. Docker DNS D. HDFS replication
   Answer: B. Reducers fetch mapper outputs through the shuffle service.

1. What does a YARN container represent?
   A. A Docker image layer B. A NameNode block C. A resource allocation for an application process/task D. A VM snapshot
   Answer: C. It packages allocated resources for execution.

1. Which YARN component is per application?
   A. ApplicationMaster B. ResourceManager C. NameNode D. DataNode
   Answer: A. Each application has its coordinating AM.

1. What does `hdfsdfs-put` do?
   A. Formats NameNode B. Lists DataNodes C. Changes replication D. Copies a local file into HDFS
   Answer: D. It uploads from client-local storage to HDFS.

1. Which command reports live DataNodes and cluster capacity?
   A. `hdfsdfs-ls` B. `hdfsdfsadmin-report` C. `jps-report` D. `dockercomposeconfig`
   Answer: B. It provides HDFS administrative report.

1. Which command inspects a file's blocks and locations?
   A. `hadoopversion` B. `hdfsdfs-cat` C. `hdfsfsck...-files-blocks-locations` D. `dockerlogs`
   Answer: C. `fsck` reports block health and location details.

1. The SecondaryNameNode is:
   A. A periodic checkpointing helper, not a hot standby B. A second DataNode C. A YARN scheduler D. A Docker profile
   Answer: A. The name does not mean automatic HA failover.

1. What does `dfs.replication` primarily set?
   A. NameNode HTTP port B. Block byte size C. Number of DNs launched D. Default replica target for new files
   Answer: D. Existing files need explicit replication changes.

1. Which command changes an existing file's replication target?
   A. `hdfsdfs-chmod` B. `hdfsdfs-setrep` C. `hdfsdfs-mkdir` D. `hdfsgetconf`
   Answer: B. `-setrep` applies to the selected file.

1. What does `dockercomposedown` normally do to TP2 bind-mounted data?
   A. Formats it B. Deletes host data directories C. Removes containers/network, not the bind-mounted host folders D. Converts it to a named volume
   Answer: C. Bind-mounted host paths persist unless separately deleted.

1. In `./data/namenode:/data/hdfs/name`, which side is inside the container?
   A. `./data/namenode` B. Both paths are inside C. Neither path is inside D. `/data/hdfs/name`
   Answer: D. The right side is the container destination.

1. What does Docker Compose `depends_on` guarantee?
   A. Startup ordering, not application readiness B. HDFS file replication C. A successful NameNode format D. Host port availability
   Answer: A. Readiness must be checked separately.

1. Why use `namenode` instead of a fixed container IP?
   A. It avoids HDFS metadata B. Docker internal DNS resolves stable service names C. It changes RPC to HTTP D. It makes a second physical host
   Answer: B. Container IPs can change after recreation.

1. What is the effect of `127.0.0.1:19870:9870`?
   A. NameNode RPC changes to 19870 B. DataNode listens on 19870 C. Ubuntu loopback port 19870 maps to container port 9870 D. HDFS block size changes
   Answer: C. The host and container ports are separate.

1. Why does TP2 use UID:GID `1001:100` ownership?
   A. To set HDFS replication B. To assign a Docker network IP C. To format the filesystem D. To let the image's Hadoop user write mounted data
   Answer: D. Host directory ownership must permit container writes.

1. Which is true of TP2's three DataNode containers?
   A. They share one VM/physical failure domain B. They are three physical machines C. They automatically provide rack diversity D. They run YARN by default
   Answer: A. Containers do not create independent hardware.

1. Which statement about `hdfsnamenode-format` is correct?
   A. It restarts only a DataNode B. It initializes a new namespace; avoid repeating on an existing cluster C. It changes the block size D. It verifies DNS
   Answer: B. Formatting can make old HDFS data inaccessible.

1. If a client uses `namenode-inexistant`, the first likely issue is:
   A. A wrong block size B. Under-replication C. DNS/endpoint resolution D. Java version
   Answer: C. The requested hostname does not resolve to the NameNode.

1. What does `hdfsgetconf-confKeydfs.blocksize` help verify?
   A. Effective value seen by that client B. Number of physical disks C. Image digest D. VM RAM
   Answer: A. It checks loaded configuration, not just file contents.

1. What does `jps` prove?
   A. Every DN is registered B. HDFS blocks are healthy C. Docker DNS works D. Which Java processes are visible to the user
   Answer: D. Follow up with HDFS-level checks.

1. Which is the best evidence that TP2 has two registered DataNodes?
   A. `dockerimages` output B. `hdfsdfsadmin-report` showing two live DNs C. `java-version` D. XML file count
   Answer: B. HDFS report confirms logical registration.

1. What is data locality?
   A. Storing only one replica B. Putting all files on the NameNode C. Scheduling computation near its data D. Binding the web UI to localhost
   Answer: C. It reduces data movement when possible.

1. Which is not one of the four core Hadoop modules?
   A. HDFS B. YARN C. Hadoop Common D. Hive
   Answer: D. Hive is a related ecosystem project.

1. Which statement about Spark is correct?
   A. It is a separate compute engine that can use HDFS/YARN B. It is the HDFS NameNode C. It is a Docker volume D. It is the same thing as Hive
   Answer: A. Spark performs computation and can run on Hadoop infrastructure.

1. Why is replication not equivalent to backup?
   A. Replication stores no data B. Live replicas may reflect deletion/corruption; backup is separately retained C. Backup is always online D. HDFS has no replica metadata
   Answer: B. Replication protects from some node failures, not logical deletion.

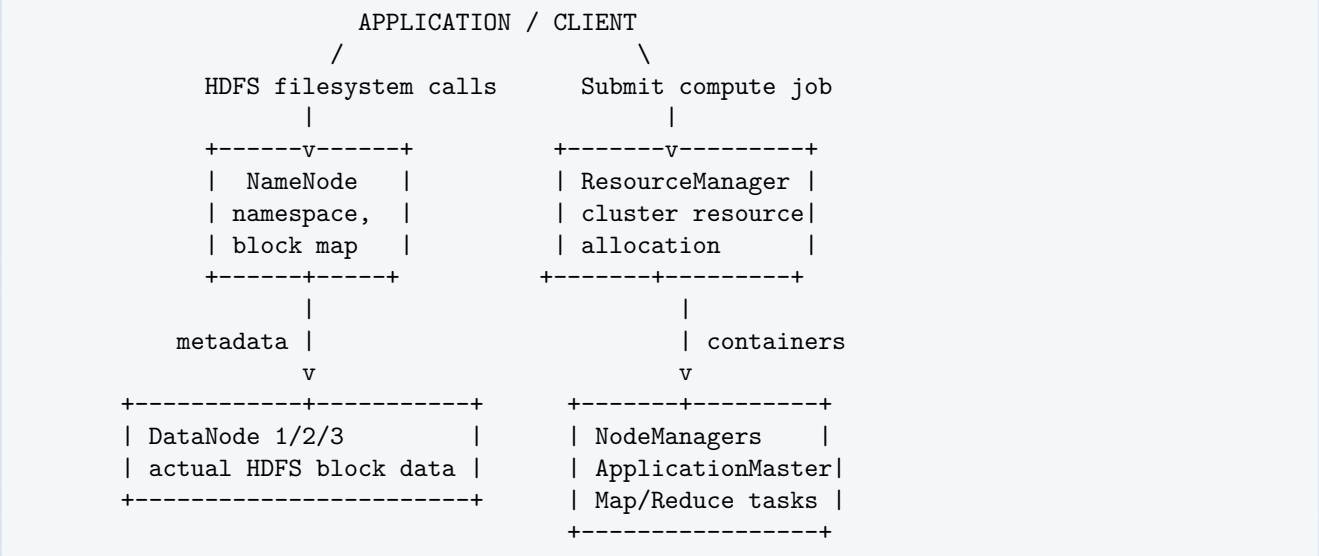
1. What is a typical HDFS workload?
   A. Millisecond updates to many tiny records B. Interactive transactional banking ledger C. High-throughput scan of large files D. Local desktop settings
   Answer: C. HDFS is optimized for large sequential access.

1. Which is a valid consequence of changing `dfs.blocksize`?
   A. It automatically restructures old files B. It renames the NameNode C. It changes Docker's host port D. It sets the default for new files; old block layouts remain
   Answer: D. Existing files keep their metadata layout.

1. Which phrase best describes MapReduce's shuffle?
   A. Group and transfer mapper outputs by key to reducers B. Copy image from Docker Hub C. Format NameNode metadata D. Replicate the final output to three VMs
   Answer: A. Shuffle routes/group intermediate key-value pairs.

Final Revision Sheets and Mock Examination

19.1 One-page Hadoop architecture summary



- HDFS: persistent distributed storage; NameNode manages metadata, DataNodes store blocks.
- YARN: schedules resources; ResourceManager is global, NodeManager is per worker, ApplicationMaster is per app.
- MapReduce: batch compute; map emits pairs, shuffle groups by key, reduce produces output.
- TP2 deploys only the HDFS side in Docker. TP1 introduces the YARN/MapReduce pseudo-distributed path.
- In TP2, all containers live inside one Ubuntu VM: logical distribution, not independent physical fault domains.

19.2 Configuration files cheat sheet

Table 19.1: Configuration files cheat sheet reference table

Configuration files cheat sheet — reference table		
File	Remember	TP example
`core-site.xml`	Common filesystem URI and other common properties	TP1 `hdfs://localhost:9000`; TP2 `hdfs://namenode:8020`

Configuration files cheat sheet — continued		
`hdfs-site.xml`	HDFS replicas, blocks, metadata/data paths and listeners	TP2 replication 2, block 8388608, name dir `/data/hdfs/name`
`mapred-site.xml`	MapReduce framework and runtime options	`mapreduce.framework.name=yarn`
`yarn-site.xml`	YARN / NodeManager settings	`yarn.nodemanager.aux-services=mapreduce_shuffle`
`hadoop-env.sh`	Shell environment/JVM settings	`JAVA_HOME`; TP2 image already has Java 17

Memory rule: core = general Hadoop filesystem setting; HDFS = storage; MapReduce = job framework; YARN = resource manager; env script = launch environment.

19.3 Essential Hadoop commands

Table 19.2: Essential Hadoop commands reference table

Essential Hadoop commands — reference table	
Goal	Command
Version	`hadoopversion` / `hdfsversion`
Format new HDFS namespace only	`hdfsnamenode-format`
Start/stop HDFS on TP1	`start-dfs.sh` / `stop-dfs.sh`
Start/stop YARN on TP1	`start-yarn.sh` / `stop-yarn.sh`
Inspect local Java processes	`jps`
List/create/upload/download/read	`hdfsdfs-ls`, `-mkdir-p`, `-put`, `-get`, `-cat`
Remove/change permission	`hdfsdfs-rm`, `-rmdir`, `-chmod`
Usage / file replication	`hdfsdfs-du-hPATH`, `hdfsdfs-setrep-w3FILE`
Cluster report	`hdfsdfsadmin-report`
File block diagnostic	`hdfsfsckFILE-files-blocks-locations`
Effective property	`hdfsgetconf-confKeyPROPERTY`

19.4 Essential Docker commands for TP2

Table 19.3: Essential Docker commands for TP2 reference table

Essential Docker commands for TP2 — reference table	
Goal	Command
Check engine/compose	`docker--version`; `dockercomposeversion`
Pull exact image	`dockerpullapache/hadoop:3.5.0`
List containers	`dockerps`; `dockercomposeps`
Validate compose syntax/model	`dockercomposeconfig`
Start initial services	`dockercomposeup-dnamenodedatanode1datanode2`

Essential Docker commands for TP2 — continued	
Start extension DN	`dockercompose--profileextensionup-ddatanode3`
View logs	`dockerlogs--tail50tp2-namenode`; `dockercomposelogs--tail80`
Run a command in a container	`dockerexectp2-namenodehdfsdfsadmin-report`
Inspect network/mounts	`dockernetworkinspectNETWORK`; `dockerinspectCONTAINER--format'{{json.Mounts}}'`
Stop/remove containers/ network but retain bind-mounted data	`dockercompose--profileextensiondown`

19.5 Ports to memorize

Table 19.4: Ports to memorize reference table

Ports to memorize — reference table		
Port	Service	Typical TP access
8020	HDFS NameNode RPC	Internal Docker bridge in TP2
9870	HDFS NameNode web UI	`http://localhost:9870` on Ubuntu VM
9866	DataNode data-transfer	Internal HDFS communication
9864	DataNode HTTP	Internal/unpublished in TP2
9000	NameNode URI in TP1 pseudo-distributed configuration	`hdfs://localhost:9000` in that example only
8088	YARN ResourceManager web UI	TP1 YARN example; not TP2

19.6 HDFS block and replication calculations

```
logical block count = ceil(file size / block size)
expected block-copy count = logical block count x replication factor
```

For TP2:

```
file = 20 MiB; block = 8 MiB; replication = 2
blocks = ceil(20/8) = 3
block sizes = 8 + 8 + 4 MiB
copies expected after convergence = 3 x 2 = 6
```

At replication 3, the same existing file needs `-setrep3`; changing the XML default alone does not modify it.

19.7 Thirty highest-value examination facts

1. Hadoop core modules are Common, HDFS, YARN and MapReduce.
2. HDFS is a filesystem; it is not a database.

3. HDFS favors throughput on large files over low-latency random updates.
4. The NameNode manages namespace metadata and block mappings.
5. DataNodes store and serve the actual block bytes.
6. HDFS clients request metadata from the NameNode and transfer bytes with DataNodes.
7. A file is divided into logical blocks.
8. The final block may be smaller than the configured block size.
9. Replication factor counts copies per block.
10. Replication adds storage overhead.
11. Replication is not backup.
12. 20 MiB at 8 MiB/block yields three blocks: 8, 8, 4 MiB.
13. At replication 2, that file expects six block copies after convergence.
14. `dfs.replication` sets the default for new files.
15. `hdfsdfs-setrep` changes a specified existing file's target.
16. `fs.defaultFS` is commonly set in `core-site.xml`.
17. HDFS data paths, block size, replication and listeners are configured in `hdfs-site.xml`.
18. `hadoop-env.sh` is a shell script, not XML.
19. `mapred-site.xml` configures MapReduce; `yarn-site.xml` configures YARN.
20. `hdfsgetconf` helps verify a loaded property.
21. `hdfsdfs` is for filesystem paths; `hdfsdfsadmin` is for administration.
22. `dfsadmin-report` checks logical DataNode status and capacity.
23. A Running container does not prove a live HDFS DataNode.
24. TP1 manual setup examples use Hadoop 3.3.6.
25. TP1 BDE2020 Docker tag uses Hadoop 3.1.3 and Java 8.
26. TP2 uses Apache Hadoop 3.5.0 with Java 17 in the image.
27. TP2 deploys HDFS only, not YARN/MapReduce.
28. TP2 RPC uses 8020; NameNode web UI uses 9870.
29. Docker Compose service DNS avoids relying on changing container IPs.
30. TP2 bind mounts persist data across container recreation; never reformat an existing namespace.

19.8 Common exam traps and misconceptions

- Saying the SecondaryNameNode is the standby NameNode.
- Saying the NameNode stores all user file bytes.
- Treating `jps` or `dockerps` as proof that HDFS is healthy.
- Confusing an HDFS path (`hdfsdfs-ls/`) with a Linux path (`ls/`).

- Treating a YARN container as necessarily a Docker container.
- Confusing a block size with a replication factor.
- Multiplying number of files by replication without first counting blocks.
- Padding the final partial block to full block size.
- Claiming replication is a backup or protects against every failure.
- Reformatting the NameNode to repair DNS, XML, port, or permission errors.
- Believing a changed XML default retroactively changes existing files.
- Using TP1's localhost:9000 for TP2's Docker URI namenode:8020.
- Claiming the TP1 BDE2020 image is Hadoop 3.3.6.
- Claiming TP2 runs YARN or MapReduce.
- Assuming three containers in one VM are three independent physical machines.
- Assuming `depends_on` means the NameNode is ready to accept requests.
- Confusing the Compose host-side port with the container-side listener port.
- Assuming container deletion always deletes bind-mounted data—or that it can never delete data if mounts were wrong.
- Treating Hive as a core Hadoop module.
- Describing Spark as a replacement filesystem rather than a compute engine.
- Assuming a correct XML file is effective without checking mounts/config directory and loaded values.
- Assuming 8 MiB is a recommended production block size; it is a TP2 teaching value.
- Treating `-setrep-w` waiting as proof of corruption; first check live DNs and capacity.
- Assuming `dockercomposedown` requires NameNode format on the next start when bind mounts persist.

19.9 Final mock examination (100 points)

Suggested timing: 90 minutes. Try without notes first; then compare against the correction.

19.9.1 Part A — Short answers (20 points; 4 points each)

1. State the roles of the NameNode and DataNode.
2. Distinguish `core-site.xml` from `hdfs-site.xml`.
3. Explain why SecondaryNameNode is not HA.
4. Distinguish `dfs.replication` from `hdfsdfs-setrep`.
5. Explain why the TP2 containers do not demonstrate three physical failure domains.

19.9.2 Part B — Calculations and interpretation (20 points)

1. A 20 MiB file is stored with 8 MiB blocks and replication factor 2. State the logical block sizes and total expected copies. (6)
2. Interpret `hdfs://namenode:8020` and explain why it works without publishing port 8020 to the Ubuntu host. (6)
3. A student changes XML replication from 2 to 3, then checks an older file and still sees 2. Explain and give the appropriate command. (4)
4. In a mount `./data/datanode2:/data/hdfs/data`, state which path is on Ubuntu and which is inside the container. (4)

19.9.3 Part C — Practical diagnosis (30 points)

1. `dockercomposeps` shows NameNode and two DataNode containers Running, but `hdfsdfsadmin-report` reports zero live DataNodes. Give a diagnostic sequence and two plausible causes. (10)
2. A student cannot open `http://localhost:9870`, and Compose uses `127.0.0.1:19870:9870`. Explain the problem and correct URL. (5)
3. HDFS client commands fail after a configuration edit. Describe how to verify the effective URI/property and whether the correct file is mounted. (7)
4. HDFS data disappears after container recreation. Give likely persistence-related causes and safe checks. (8)

19.9.4 Part D — Conceptual essay (30 points)

1. Explain the relationship between HDFS, YARN and MapReduce during a batch job. Include the role of data locality. (12)
2. Compare standalone, pseudo-distributed and fully distributed Hadoop modes, including what TP1 and TP2 actually exercise. (10)
3. Explain why replication improves some forms of fault tolerance but is not a backup or complete HA solution. (8)

19.10 Mock examination correction and marking guidance

1. **NameNode:** namespace/metadata/block mapping and coordination. **DataNode:** block storage, client data service, heartbeat/block report. Award 2 points per clear role.
2. **core-site:** common setting/default filesystem URI. **hdfs-site:** HDFS blocks, replicas, storage paths, listeners. Award 2 per distinction/example.
3. SecondaryNameNode periodically checkpoints metadata; it is not a synchronized standby taking over on failure. HA needs active/standby and failover/shared edits. Full marks for "checkpoint, not failover."
4. XML `dfs.replication` is the default for new files; `-setrepNpath` changes the specified existing file's target. Example `hdfsdfs-setrep-w3/tp2/blocks/20MiB.bin`.

5. All containers share one Ubuntu VM, kernel, virtual disk and physical host; one host/VM failure can affect all.
6. `ceil(20/8)=3`; sizes 8+8+4 MiB; six expected copies at replication 2. 2 points blocks, 2 sizes, 2 copies.
7. Scheme is HDFS, host is Docker service `namenode`, port 8020 is RPC. Compose bridge peers reach it internally; host port publishing is only needed for access from outside that network. Do not confuse with 9870 HTTP.
8. Existing file retains its replication target; use `hdfsdfs-setrep-w3/path/file`. XML default applies to new files.
9. `./data/datanode2` is the Ubuntu-side bind source; `/data/hdfs/data` is the container-side destination.
10. First verify logs (`dockerlogs`), report again after startup, `getenthostsnamenode`, network membership, XML/effective `fs.defaultFS`, and data-directory ownership/mounts. Plausible causes include bad NameNode hostname/port, network mismatch, DN startup/XML error, or permissions. Do not format as generic repair.
11. The host port is 19870, so use `http://localhost:19870`; the process remains on container port 9870. Alternatively restore host mapping to 9870.
12. Run `hdfsgetconf-confKeyfs.defaultFS` and relevant property in the client container; inspect `dockerinspect` mounts for the expected config file, check `HADOOP_CONF_DIR`, duplicate property and logs; recreate/restart affected containers after correction.
13. Check that NameNode and each DataNode data directory is a bind mount to the intended persistent Ubuntu project path, correct working directory/project, and that directories were not removed. Inspect Docker mount `Source/Destination` and host `data/`; preserve any remaining files and do not format.
14. HDFS provides input/output blocks; YARN allocates containers across NodeManagers; MapReduce maps input records, shuffles/groups by key, reduces and writes output. Scheduling near blocks is data locality, reducing network transfer when possible.
15. Standalone: local process/local FS; pseudo: separate daemons on one host; fully distributed: services across multiple hosts. TP1 shows standalone and pseudo-distributed (and optional BDE2020 Hadoop 3.1.3 Docker); TP2 uses Apache 3.5.0 HDFS containers inside one Ubuntu VM, not a fully independent physical cluster.
16. Replication provides alternate copies for certain disk/DataNode failures, but consumes storage and can propagate logical deletion/corruption; backups are separately retained. Data replication does not make NameNode metadata automatically highly available; HDFS HA requires an HA control-plane design. Award for data copies, limitations, and backup/HA distinction.

One-page Big Data summary

- **Big Data:** data/workloads whose scale, arrival rate, diversity, quality or latency needs exceed practical single-system methods.
- **5 Vs:** Volume (amount), Velocity (arrival rate), Variety (formats), Veracity (quality/uncertainty), Value (useful outcome).
- **Data types:** structured (tables), semi-structured (JSON/XML), unstructured (images, audio, free text).
- **Distributed computing:** coordinate networked machines; parallelism runs independent partitions simultaneously.
- **Scale-up vs scale-out:** add resources to one machine vs add workers/nodes.
- **Batch vs stream:** bounded job over stored data vs ongoing low-latency event processing.
- **Reliability:** replication tolerates some node failures; it is not an independent backup. Data quality, security and governance remain essential.
- **Applications:** IoT/robotics telemetry, predictive maintenance, manufacturing, healthcare, finance, telecom and web logs.

Exam answer pattern

Define the property, identify its engineering challenge, give a concrete data example, and state why distribution helps (or when its overhead is not justified).

One-page Hadoop architecture summary

- **Hadoop Common:** shared utilities/libraries.
- **HDFS:** distributed file storage. NameNode manages namespace and block metadata; DataNodes store/serve blocks.
- **YARN:** resource allocation. ResourceManager is cluster-wide; NodeManager is per worker; ApplicationMaster coordinates one application; containers carry resources.
- **MapReduce:** batch computation: input splits, map key-value output, shuffle/sort by key, reduce, write output.
- **Data locality:** schedule compute near input blocks when possible to reduce network transfer.
- **TP1:** Hadoop 3.3.6 standalone/pseudo-distributed; XML config, SSH, HDFS/YARN and example MapReduce. Optional BDE2020 image is Hadoop 3.1.3/Java 8.
- **TP2:** Apache Hadoop 3.5.0/Java 17 in one Ubuntu VM; one NameNode and two DataNodes initially; HDFS only.

One-page HDFS summary

- A file is split into logical blocks; each block may have several replicas on DataNodes.
- The NameNode manages namespace/block mappings; DataNodes store actual bytes and send heartbeats/block reports.
- Read: client asks NameNode for block locations, then reads from a suitable DataNode. Write: NameNode allocates targets; client streams data through a DataNode pipeline.
- DataNode failure: missed heartbeat, alternate replica reads, and re-replication when possible. NameNode failure without HA interrupts namespace operations.
- SecondaryNameNode checkpoints metadata; it is not automatic failover. HDFS HA is a separate design.
- TP2 calculation: 20 MiB / 8 MiB gives blocks 8+8+4 MiB = 3 logical blocks. Replication 2 means 6 block copies; no padding of the final partial block.
- Format the NameNode once for a new namespace only. Never reformat to fix an endpoint/configuration error.
- HDFS fits large sequential files and throughput; small-file/random-low-latency workloads are less suitable.

One-page Hadoop configuration summary

- `core-site.xml`: common defaults; `fs.defaultFS` selects the default filesystem URI.
- `hdfs-site.xml`: HDFS blocks, replication, NameNode/DataNode paths and service listeners.
- `mapred-site.xml`: MapReduce settings; TP1 uses `mapreduce.framework.name=yarn`.
- `yarn-site.xml`: YARN settings; TP1 enables NodeManager `mapreduce_shuffle`.
- `hadoop-env.sh`: shell/JVM environment such as `JAVA_HOME`; not XML.
- TP1: `hdfs://localhost:9000`, replication 1, local paths. TP2: `hdfs://namenode:8020`, replication 2, 8 MiB blocks, mounted container paths.
- Confirm effective values with `hdfs getconf -confKey ...`; check mount path, `HADOOP_CONF_DIR`, duplicate properties and logs.
- Changing a mounted config normally requires recreating/restarting affected processes. XML defaults do not rewrite existing file metadata.

One-page Docker and TP2 summary

- Physical host → VirtualBox → one Ubuntu VM → Docker Engine/Compose → bridge network and Hadoop containers.
- TP2: Apache Hadoop 3.5.0, Java 17 in image, one NameNode and two DataNodes initially; optional third DataNode with Compose profile. No YARN/MapReduce/SecondaryNameNode/HA/EC.
- Internal DNS uses service names such as `namenode`; host/container IPs can change after recreation.
- Ports: 8020 RPC (internal), 9870 NameNode HTTP UI, 9866 DataNode data transfer, 9864 DataNode HTTP. A Compose mapping controls host access separately.
- Bind mount `./data/namenode:/data/hdfs/name`: left is Ubuntu, right is container. HDFS XML paths must match the container path.
- Ownership UID:GID 1001:100 permits the Hadoop user to write to mounted directories; it is distinct from HDFS `chmod`.
- `compose down` removes containers/network, not ordinary bind-mounted files. Preserve `data/`; format NameNode only once.
- Running container is not proof of HDFS registration: check logs and `hdfs dfsadmin -report`.

One-page HDFS and Docker commands summary

Command	Purpose
`hdfs dfs -mkdir -p /path`	Create an HDFS directory tree.
`hdfs dfs -put file /path/`	Upload a client-local file to HDFS.
`hdfs dfs -get /path/file .`	Download from HDFS to local storage.
`hdfs dfs -ls /path; -cat /path/file`	List paths or display a small text file.
`hdfs dfs -rm [-r] /path`	Remove a file or (with `-r`) directory tree; destructive.
`hdfs dfs -du -h /path; -df -h /`	HDFS path usage and HDFS filesystem capacity.
`hdfs dfs -chmod 755 /path; -chown u:g /path`	Change HDFS permissions/ownership; privileges apply.
`hdfs dfsadmin -report`	Live/dead DataNodes and cluster capacity.
`hdfs getconf -confKey dfs.blocksize`	Read the effective client configuration value.
`hdfs dfs -setrep -w 3 /file`	Change an existing file's replica target and wait for convergence.
`hdfs fsck /file -files -blocks -locations`	Inspect file blocks, health and DataNode locations.
`docker compose ps; docker logs NAME`	Container state and service logs.
`docker compose -profile extension up -d datanode3`	Start TP2's optional third DataNode.

Focused Two-Day Revision Strategy

20.1 Day 1 — Understand the system

1. **60 min — Big Data:** learn the 5 Vs, data types, scaling, distributed/parallel processing, and batch vs stream. Explain each aloud with one example.
2. **75 min — Hadoop architecture:** memorize four core modules; draw HDFS/YARN/MapReduce boundaries from memory.
3. **90 min — HDFS:** learn NameNode/DataNode, metadata vs blocks, read/write sequence, failure behavior, replication, SecondaryNameNode, and HA distinction.
4. **45 min — Block arithmetic:** repeat the 20 MiB / 8 MiB / replication 2 and 3 calculations without notes; explain the partial final block.
5. **60 min — YARN/MapReduce:** draw RM/NM/AM/containers; work WordCount and identify the network shuffle.
6. **60 min — Configuration:** rewrite the five-file table from memory; contrast TP1 `localhost:9000` with TP2 `namenode:8020`.
7. **20 min — Retrieval practice:** answer theoretical questions Q5, Q7, Q10, Q14, Q18, Q21, Q27 and Q33 without looking.

20.2 Day 2 — Apply and retain

1. **60 min — TP1/TP2 comparison:** list versions, topology, components installed, ports, paths, and distinct roles.
2. **60 min — Commands:** classify each command as Linux, HDFS filesystem, HDFS administration, or Docker; practice explaining `put`, `get`, `report`, `fsck`, `getconf` and `setrep`.
3. **75 min — Docker deployment:** draw VM/container/network/mount layers; explain UID:GID ownership, ports, DNS and persistence.
4. **60 min — Troubleshooting:** use the diagnostic table; for each fault name two checks and one non-destructive correction.
5. **45 min — Definitions and comparison bank:** use flashcards; focus on the common traps.
6. **90 min — Mock examination:** sit the mock closed-book, then score it and revisit missed concepts.
7. **Final 30 min — One-page review:** ports, XML files, block arithmetic, TP versions, format warning and 30 facts.

Priority under severe time pressure: HDFS roles and failure behavior; 20 MiB block calculation; config files/effective properties; TP1 versus TP2 version/scope; ports and Docker DNS; default replication vs `-setrep`; persistence and the one-time format warning.

References

21.1 Assignment sources

1. *TP1 — Installation et Prise en Main de Hadoop*, Pr. ALAMI Nabil, Universite Mohammed 1er, ENIAD Berkane, academic year 2026/2027. Primary evidence for Hadoop 3.3.6 manual installation, pseudo-distributed configuration and the BDE2020 Hadoop 3.1.3/Java 8 Docker example.
2. *TP2 — Ingenierie Big Data: Deploiement et configuration avancee de Hadoop dans une VM Ubuntu*, Pr. ALAMI Nabil, academic year 2026/2027. Primary evidence for Apache Hadoop 3.5.0, Java 17, HDFS-only Docker Compose topology, configuration, ports, mounts, block experiment and extension DataNode.

21.2 Additional official references

1. Apache Hadoop 3.5.0 documentation: <https://hadoop.apache.org/docs/r3.5.0/>
2. Apache Hadoop HDFS Architecture: <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>
3. Apache Hadoop Single Node Setup (standalone and pseudo-distributed examples): <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/SingleCluster.html>
4. Apache Hadoop YARN: <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html>
5. Apache Hadoop Docker documentation: <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/HadoopDocker.html>
6. Docker Engine installation for Ubuntu: <https://docs.docker.com/engine/install/ubuntu/>
7. Docker Compose documentation: <https://docs.docker.com/compose/>

How to use sources: The assignment-specific versions, commands and topology above follow TP1/TP2. General theory is supplemental and should not be mistaken for a claim that the corresponding service ran in TP2. When exact behavior depends on Hadoop release or deployment configuration, verify against the relevant versioned Apache documentation.

Final exam habit: State the component, its responsibility, the relevant configuration/command, and the expected evidence. For TP2, always distinguish what is inside the container, what persists on Ubuntu, what is reachable only on the Docker network, and what is actually deployed.